SWMAL_Grp10-4

November 27, 2025

1 Aarhus Universitet — SWMAL E25

1.1 Assignment O4

1.1.1 Gruppe 10

1.1.2 Members

Name	Student ID
Per Hindbo Clausen	202007348
Mads Kampmann Elkjær	202209870
Daniel Them	201809623
Frederick Lykkegaard Nielsen	202207768

1.1.3 Hand-in Date

04/12-2025

2 Introduction

This report presents our final end-to-end machine learning project for O4. Our goal is to build an image-classification system that predicts the breed of a cat from a photo. We will focus on five breeds, Bengal, Domestric Shorthair, Maine Coon, Ragdoll and Siamese. We want to design and implement a pipeline, so that new breeds can be added with minimal changes.

We use the Kaggle Cats Breed dataset, which gives us real world images of five different breeds with variation in resolution, lighting and background.

In the report, we follow an end-to-end structure, which defines the problem, describe the dataset, our choice of machine learning methods, explaining how we process, train, and evaluate our model. We will also discuss performance metrics, risks of under- and overfitting, and the optimizations used to improve generalization. The report concludes with key results and reflections on possible future improvements.

3 Problem statement

The goal is to develop a machine learning model that can identify the breed of a cat from an image. Correctly recognizing car breeds is a task that involves understanding visual features such as fur pattern, colors, shape and orhter characteristics. However, these features could possibly be subtle and difficult to distinguish, especially on breeds that look alike.

The main challenge of real-world images, is that they can vary in lighting, resolution, pose and background, which makes it harder for a model to focus on the important features of the cat, instead of background and lighting features. Our approach includes preparing and cleaning the dataset, selecting appropriate machine learning methods, training the model and evaluation its performance using relevant metrics. We want to create a model that can somewhat reliably predict the five target cat breeds and can be expanded to include more breeds in the future.

4 Dataset

4.0.1 a. Concept

We want to build a robust image-classifier that predicts a cat's breed from a single photo. The initial scope covers five breeds—Bengal, Domestic Shorthair, Maine Coon, Ragdoll, and Siamese, but the architecture will be modular (Each cat breed has their own folder) so adding new classes later requires only dropping more folders into the dataset, re-running the split and re-training or fine-tuning. We normalize inputs consistently, apply conservative augmentations to improve generalization (flip/rotate/brightness/contrast/zoom), and track performance with accuracy and macro-averaged F1. To understand model behavior, we'll use a confusion matrix and per-class metrics; this is especially useful for visually similar breeds or when classes are imbalanced. The end product is a saved model artifact plus a small script that takes an image path and returns top-1 prediction with confidence, and maybe top-k probabilities for transparency.

4.0.2 b. Dataset

We use the Kaggle Cat's Breed Dataset, which provides exactly the five classes we target and already follows a directory-per-breed structure. This structure is compatible with standard loaders fx. (Keras `image_dataset_from_directory`) and plays nicely with reproducible train/validation/test splits. The pipeline stays open to growth: if we later collect more images for these breeds or introduce new breeds, we simply add folders with the appropriate names and rerun the preprocessing/splitting steps. This makes it straightforward to scale both the data and the classification without rewriting code.

4.0.3 c. Dataset description

Each sample is a color photograph (commonly JPEG/PNG in RGB) with non-uniform resolution and aspect ratio. Labels are implicitly defined by folder names, which reduces the chance of metadata drift but can still contain occasional label noise (mis-sorted images). In a quick exploration pass, we verify the directory layout, count the images per class, and check for corrupt files or near-duplicates using a perceptual hash. We look at global summaries like width/height distributions and aspect ratios to inform the target input size and resizing policy (e.g., pad/crop vs. stretch). We also visualize single-image RGB histograms and small grids of random examples per class to spot systematic differences such as background bias (studio shots vs. household environments), lighting

extremes, or watermark artifacts. To hold the scope of the project down we won't go through the pictures for background bias, as i might take a lot of time. There might be some issuese which could be mild class imbalance, resolution variance, and varied backgrounds that can mislead the model into learning context instead of the cat.

4.0.4 d. Use of dataset

We'll use the dataset for multiclass classification: given a cat photo, predict one breed among our five classes. Classification fits because the target is a discrete label, not a numeric value (so not regression). We'll resize and convert to RGB to standardize inputs and stabilize training, then make stratified train/validation/test splits to get fair evaluation. For metrics, we pick accuracy for overall performance and macro-F1 so small classes count equally (We might add a cat breed with not a lot of pictures). We will use a confusion matrix that shows us which breeds the model confuses.

4.0.5 Sources:

- Kaggle Cat's Breed Dataset
- scikit-learn: train_test_split (stratified splitting)
- scikit-learn: StratifiedKFold
- scikit-learn: StratifiedShuffleSplit
- scikit-learn: StratifiedGroupKFold
- scikit-learn: f1_score (use `average="macro"`)
- scikit-learn: confusion_matrix
- scikit-learn: ConfusionMatrixDisplay
- scikit-learn: classification_report
- scikit-learn: cross_validate (multi-metric CV)
- scikit-learn User Guide: Metrics & scoring (classification metrics)
- scikit-learn User Guide: Cross-validation

We start by setting the dataset path, importing the libraries, and configuring plotting so the rest of the notebook knows where the images are and can visualize results consistently.

[1]:
```
# Setup: Datapath and imports
DATA_DIR = "CatData/cat_v1"

from pathlib import Path
import numpy as np
import pandas as pd
from PIL import Image, UnidentifiedImageError
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (8,5)
plt.rcParams["axes.grid"] = True
```

Next, we scan the class folders, open each image, and record format, color mode, width, height, and aspect ratio to build a simple metadata table.

[2]:
```
# Scan dataset (formats, color spaces, sizes)
IMG_EXTS = {".jpg",".jpeg",".png",".bmp",".gif",".tif",".tiff",".webp"}

rows, bad = [], []
root = Path(DATA_DIR)
classes = sorted([d.name for d in root.iterdir() if d.is_dir()])

for cls in classes:
    for p in (root/cls).rglob("*"):
        if p.is_file() and p.suffix.lower() in IMG_EXTS:
            try:
                with Image.open(p) as im:
                    im.load()
                    w, h = im.size
                    rows.append({
                        "path": str(p),
                        "label": cls,
                        "format": im.format,     # JPEG, PNG, ...
                        "mode": im.mode,         # RGB, L, RGBA, ...
                        "width": w,
                        "height": h,
                        "aspect": (w/h if h else np.nan)
                    })
            except (UnidentifiedImageError, OSError):
                bad.append(str(p))

meta = pd.DataFrame(rows)
print(f"Classes: {classes}")
print(f"Total images: {len(meta)}  |  Unreadable: {len(bad)}")
meta.head()
```

```
Classes: ['bengal', 'domestic_shorthair', 'maine_coon', 'ragdoll', 'siamese']
Total images: 3649  |  Unreadable: 0
```

[2]:

	path	label	format	mode	width	height	aspect
0	CatData/cat_v1/bengal/81cfdf3adf43735cfb1ff26f…	bengal	JPEG	RGB	1200	800	1.500000
1	CatData/cat_v1/bengal/Bengal_35.jpg	bengal	JPEG	RGB	300	306	0.980392
2	CatData/cat_v1/bengal/00e79f939696ea0c09560315…	bengal	JPEG	RGB	2554	3222	0.792675
3	CatData/cat_v1/bengal/boy-bengal-cat-names.jpg	bengal	JPEG	RGB	1250	1351	0.925241
4	CatData/cat_v1/bengal/Bengal_197.jpg	bengal	JPEG	RGB	300	225	1.333333

Then we print a directory tree to document how the data are organized on disk, showing the root and the class subfolders.

[3]:
```
# Representation on disk (folder structure)
def print_tree(path: Path, max_depth=1):
    def _walk(p, prefix="", depth=0):
        if depth > max_depth: return
        entries = sorted(p.iterdir(), key=lambda x: (not x.is_dir(), x.name.
 ↪lower()))
        for i, e in enumerate(entries[:50]):  # truncate for readability
            is_last = (i == len(entries)-1)
            branch = "   " if is_last else "   "
            print(prefix + branch + e.name + ("/" if e.is_dir() else ""))
            if e.is_dir():
                _walk(e, prefix + ("    " if is_last else "    "), depth+1)
    print(p.name + "/")
    _walk(path)

print_tree(root, max_depth=1)
```

```
1XAG3BWR6R6Y.jpg/
  bengal/
      00438-Bengal-cat-snarling.jpg
      00441-Very-timid-Brown-Spotted-Bengal-cat-ears-back.jpg
      00e79f939696ea0c095603154c4af840.jpg
      03915-Pair-of-Bengal-kittens-white-background.jpg
      04413-Bengal-cat-white-background.jpg
      047ffa2f7165ccc4585e418ddb800299.jpg
      05050-Brown-spotted-Bengal-cat-sitting-on-grey-background.jpg
      09a0c5d574f818471c84cf509fc786dc.jpg
      0c87060cea587a01f0a254066c95deaa.jpg
      106f59608bc9fc6ac11df31008efdf94.jpg
      1096165.jpg
      1111246.jpg
      1113154.jpg
      1142194.jpg
      119mokiL1.jpg
      1200px-Bangie_the_Bengal_Cat.jpg
      122433177_3649288731759742_6202950879910256561_o.jpg
      12651963_f520.jpg
      14105889_f520.jpg
      15658-Brown-spotted-Bengal-female-cat.jpg
      15a0a84bd11fcae601ed41f5672191ba.jpg
      1aeae43be56c47450ef4a3354d3e940d.jpg
      1b2a68f85a32410fb3fd776927a2f4ef.jpg
      1e8351aa322b32f7a23980d7cdbed1a8.jpg
      20121130172827.jpg
      22384097_10154807489365927_7732210437813358666_o.jpg
```

```
      2308642374_57489bb2ce_o-57b742c15f9b58cdfdb97b03.jpg
      2385251605b4059a955e526.09727891-profile-1422.jpg

28157986_217164348841297_8045674480825008128_n-5a9606581f4e1300367fff3f.jpg
      28209b6d36d53298996458299227dca4.png

28430549_572162953143506_2215371262585208832_n-5a96060b1f4e1300367ff4f0.jpg
      306059656_c4fafc4a87_o.jpg
      32249-Bengal-female-cat-white-background.jpg
      379800.jpg
      3931eb090308eeaa58719fe66b9926c3.jpg
      3be35b792cbaa987776772fd2a67b78e.jpg
      3eb7e9a7542bdead2c633e4a2b06a588.jpg
      40706-Portrait-of-Bengal-cat.jpg
      40725-Bengal-cat-sitting-white-background.jpg
      425c7c6eba0b92a7da8d32e4502ff0e5.jpg
      42cfe991ab15fe999d446660b4ccb0b8.jpg
      53653d0ade9fc4d625055cd5c45b8d5c.jpg
      5fba2bc61c502d3177e845b48807e38d.jpg
      60a954c22bba1.jpg
      610850dbd28f3cfea67e7d6350a8a0c1.jpg
      677597.jpg
      67dd59a71ac467ca3224dd3ce87b1a6e.jpg
      687362.jpg
      68a0ecf22205aca18f080c83a32e083e.jpg
      6ae66c9e98081d48648b8ecc9784aedf.jpg
  domestic_shorthair/
      046dce16530c0e2f62348adc724dbc4b.jpg
      04f2e5e2f67443c7c3299ce4078dc2a2.jpg
      0C40F888-7B22-4B2C-88B5-049BBB40E374.jpeg
      0cd53884802ed1f687f6ddb860973b8b.jpg
      10bc40c87f24b7442f454f90f1248e16.jpg
      1200-93358197-domestic-shorthair-cats-playing.jpg
      1200px-American_Shorthair.jpg
      1200px-Cat03.jpg
      1200x0 (1).jpg
      1200x0 (2).jpg
      1200x0 (3).jpg
      1200x0.jpg
      1280-594481594-little-kitten-scaled.jpg
      1286px-British_Shorthair_Cat.jpg
      13299451414_6217c5c653_b.jpg
      1585003597_2615038995e793c4deb2bb.jpeg
      17e3677e829dc216edcc4c1982680495.jpg
      189edc1cdd4d2ad45352e6d5a05ebdf2.jpg
      19072-British-Shorthair-tabby-tortoiseshell-cat-reclining-white-
background.jpg
      19c64d2430d89fb1d0b21f3037b52431.jpg
```

```
      20-02-2020RG-3-scaled.jpg
      20171217_153823.jpg
      20180808_221155.jpg
      204574584.jpg
      204646681.jpg
      210084761.jpg
      228794488.jpg
      232069151.jpg
      242111_e8cda_orig.jpg
      31f7662ca0182a2ef145e580745a1a01.jpg
      37087816_10216998859910320_332375493129011200_n.jpg
      388271506.jpg
      404469246.jpg
      438649_28986_orig.jpg
      461138744.jpg
      466748758.jpg
      482497577.jpg
      499844134.jpg
      501189109.jpg
      502158847.jpg
      504925860.jpg
      508396792.jpg
      509787583.jpg
      50f20a3651b0742f2eb623a5c011a653.jpg
      523771009.jpg
      536180_dd3d3_orig.jpg
      543426094.jpg
      546250_9f2e0_orig.jpg

5bdf4ba6-184b-4559-88e3-8cabc3b9d89a_5.a15ed7d25a297e1bf4776d6d5691a422.jpeg
      5bec89c5926a1.image.jpg
  maine_coon/
      04451e0620bad89784005f3443dd7fed.jpg
      06a71c9902b7dcbb9f741d0599baf907.jpg
      098d3da3d30a35356744d87c57af4280.jpg
      11-25.jpg
      1280-491043662-tabby-maine-coon-cat.jpg
      14052241_f120.jpg
      14134300_f1024.jpg
      154d0dfcc8a953dc2adea273e02c0cb2.jpg
      1692_191118161233Q7_104.jpg
      17826-Maine-Coon-cat-white-background.jpg
      2003-4288-2848-dsc-8088-2e700.dsc-8088.htm
      20180512_031837-e1530464710292.jpg
      21667-Maine-Coon-kitten-white-background.jpg
      21911f4f1af72470cdf93bcfe4ca4591.jpg
      250a50bcbb61c258523571a78c63b787.jpg
      2790ae2c5f7e4ff5e12346fe0eb2b4ab.jpg
```

```
    28385-Maine-Coon-kitten-white-background.jpg
    29769a4f1c7ddf57d81b5d4bfbd084b4.jpg
    2sv63h8w3pt21.jpg
    30859230298_aea362e94d_b.jpg
    3506c008f54e869aa77e405e089550b5.jpg
    36f7f53a0f1b70a126167f31f7a114ca.jpg
    39728-Tabby-Maine-Coon-cat-white-background.jpg
    39730-Tabby-Maine-Coon-cat-white-background.jpg
    3a8208778a3718a73723b93768cfd8da.jpg
    3adb3fe3adb0ab2e044e6e3ba2469a54.jpg
    422600045dc93960e3ef2feb104f56b0.jpg
    42e8f81f485608e6899316ccac139a20.jpg
    45aaec1146c25dac8c832190e6057401.jpg
    4912015294ef862a23196b23de8a17d6.jpg
    4c67d602534c656db0997a7bdc72c276.jpg
    4d7499c04dcc3706dbbc4d95be2fed9e.jpg
    4fc046f9b86645a27c1de1207f1dc2e5.jpg
    502154.jpg
    51016dfcc8b60bb708b34ba27b92a051.jpg
    5a40909340d5251a5d24df5fca8ff16d.jpg
    5d06a126825383a5725a7251f747cdfd.jpg
    5d816695a728c4d4b49306719951c29f.jpg
    5ea0417ae1b850f3450b0a1420c6a214.jpg
    5EEC667B-3B20-46B2-A162-ABDEEC075F69.jpeg
    69a5c257eede39e548ccbb4b23006387.jpg
    6a791bc868d7942b58d2feea5c3c8894.jpg
    6b33dcc2c854f54de0ae6f6f8ff72eaf.jpg
    6e4833371387866c23c15259c60f10f0.jpg
    7728082365d689027aa2c52.10203604-breed-344.jpg
    813c64fa4a26a08852379a5261956886.jpg
    84a0996fb6460e2249187fba9f125fee.jpg
    84c440da9f7f78bdb8bc94b6ca6c332c.jpg
    85011c8f18fd41087cab8f7fbf84419b.jpg
    854d769e2b670e1f95d06ee690d411bb.jpg
ragdoll/
    04ecca85dae9ae1f5f404e68a289139b.jpg
    0599222d68f89df6a17784997ad4c816.jpg
    0a8d33cd127ba447efd93b9488585790.jpg
    0a9df4c05aff675b1426c701fea9e0dc.jpg
    0c8e50eb24db1849c1a0ecc638b9ba5b.jpg
    11213-Tortoiseshell-and-two-ragdoll-cats.jpg
    11e43d6d946b44abc868901dbdfdc7b5.jpg
    1200-37427470-ragdoll-cat.jpg
    124199014_xl.jpg
    1732624227ee9a0bb58e702d6a2d9f3f.jpg
    181124608_a18f0d495e_b.jpg
    19cf174d39b6e284546bd3c9dde2617d.jpg
    1ef5b49ffff19251b56c7786486f84ba.jpg
```

```
      1fa7036535595c67b52c0bb57764d838.jpg
      20648-Lilac-bicolour-Ragdoll-cat-lying-with-head-up.jpg
      21bcf0a08369291d071821a629b60258.jpg
      22219924_719461648244213_4942048061401934211_o.jpg
      2751140.png
      27665-Blue-eyed-Ragdoll-kitten-white-background.jpg
      276bdce103d8590fd3ebc2cf7acf1ba6.jpg
      2cf008151de12819f4ebad7e5c24a266.jpg
      2e2cc888-7cdd-6404-f139-79c1f201d23b.jpeg
      2e3f652667c93ac2dd4c5eb2760c93fe.jpg
      38773-Ragdoll-kitten-among-spring-flowers.jpg
      39421-Blue-eyed-Ragdoll-kitten-white-background.jpg
      39772-Ragdoll-cat-white-background.jpg
      3a189f490f7dd30e3a83a8f677523c9e.jpg
      3b8a614226a953a8cd9526fca6fe9ba5.jpg
      3fe47f4244148fcdc9cfd8d8c3e44dda.jpg
      41534-Ragdoll-kitten-10-weeks-old-white-background.jpg
      43ebf72b13de09361d7a52878773d.jpg
      49b826e113fb2acea171222930648c5c.jpg
      49dd1d27a854c4b44b72932aa208d04e.jpg
      4dfc69125aa9cfafb5c7333ac2211b6e.jpg
      549162.jpg
      56902602_414038916065155_4512174701024918698_n-
95d7a6c3150e46be804a19a821abeceb.jpg
      5ab4f4dbd3b6a5fb3ff4af08a31f0dbf.jpg
      5be919c106a1a2c4681972f940121a9a.jpg
      5dbd9785be54163365380837decb7b77.jpg
      60841aa3bf294bb6bcb203a0e8d9a191.jpg
      63d5612af077b1d2f62687467860bdd4.jpg
      68636f1eed8430aa99bcbc300eea98bf.jpg
      69d6575461a0abe33a995c7792335.jpg
      6a02fbf286bc2e5cde7064dc0e28d8c5.jpg
      6e5a57aadfb71f4635e6f99956a14772.jpg
      72b906a708de17740617e3d907b802d2.jpg
      741582.jpg
      7536708_XXL.jpg
      76911a3bb79a442c891d113e0156e086.jpg
      77466c59fdd381902793db0f5ac22ec5.jpg
  siamese/
      01FUL1T2FSY8.jpg
      02BQ9G8GRJ5G.jpg
      02NWWAHTQGXW.jpg
      02V221B4MR5L.jpg
      032VXIEAY2ND.jpg
      04QFCY4SO75A.jpg
      059QWE3U7U3Q.jpg
      059Y1KGDXHXA.jpg
      06BN2IHHON35.jpg
```

```
06C3ICEWJQ7Y.jpg
0706f1a90468dec8fe5f2d00f1a33f4e.jpg
07N4L8J7VX74.jpg
08J1O4YWFY6A.jpg
09ZV6J8EK49K.jpg
0_GLP_SAH_SM190278_30JPG.jpg
0a92a43b384b5f41931afa751601210f.jpg
0AG1KKQIHG0C.jpg
0AHTZA10ODLH.jpg
0B76UT4VATNW.jpg
0CMMOVEG1RYP.jpg
0CVW8UTW8XPB.jpg
0FMQHNWS8YQR.jpg
0FV04P1M1Z89.jpg
0FZOAPF3624B.jpg
0GOPQREKPXFB.jpg
0GUVA5EEJ10H.jpg
0HQAP0YUTXDX.jpg
0INN99LL81SY.jpg
0IWHBN5CMMRU.jpg
0KD7O4S81NDD.jpg
0LXYEI4K6VYA.jpg
0M69ENIUV16E.jpg
0MJOBNRCPJF1.jpg
0NVHB58JVZIG.jpg
0O9RHU81QW6V.jpg
0OP58JDO3N5L.jpg
0POVPJGPPPYV.jpg
0PVBVWCK47KG.jpg
0QZYSU91SKQR.jpg
0RVEY6ADRB1X.jpg
0S7B8Z7LV5WL.jpg
0T28H1DM8366.jpg
0T9NIKLALJ2S.jpg
0TGPDFM0MW4F.jpg
0UE1RTPH5658.jpg
0UZXYIZB60T3.jpg
0WDEQ7ANHIYY.jpg
0YLAMIS1EQOR.jpg
0YPCK3P9ANI7.jpg
0YQTSNNN3XGB.jpg
```

After that, we show class counts and simple tables of image formats and color modes. This checks how balanced the classes are and what kinds of files we have.

[4]:
```
# Basic tables
# Class counts
cls_counts = meta["label"].value_counts().sort_index()
```

```
display(pd.DataFrame({"count": cls_counts}))

# Formats & color spaces
display(meta["format"].value_counts().to_frame("count").rename_axis("format"))
display(meta["mode"].value_counts().to_frame("count").rename_axis("mode"))
```

```
                    count
label
bengal                527
domestic_shorthair    520
maine_coon            540
ragdoll               557
siamese              1505
```

```
        count
format
JPEG     3639
PNG         7
MPO         2
WEBP        1
```

```
      count
mode
RGB    3644
RGBA      2
P         2
L         1
```

We then compute size stats per class and plot histograms of width, height, and aspect ratio. This tells us how big images are and helps if we want to choose a standard resize.

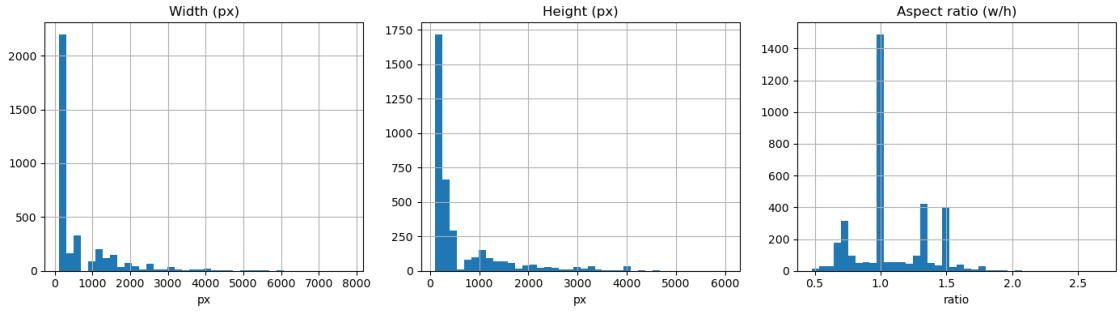
[5]:
```
# Size statistics + histograms (width/height/aspect)
size_stats = (meta
    .assign(megapix=(meta["width"]*meta["height"])/1e6)
    .groupby("label")[["width","height","aspect","megapix"]]
    .agg(["count","min","median","mean","max"]))
display(size_stats)

fig, axes = plt.subplots(1,3, figsize=(14,4))
axes[0].hist(meta["width"],  bins=40); axes[0].set_title("Width (px)"); ␣
 ↪axes[0].set_xlabel("px")
axes[1].hist(meta["height"], bins=40); axes[1].set_title("Height (px)");␣
 ↪axes[1].set_xlabel("px")
axes[2].hist(meta["aspect"], bins=40); axes[2].set_title("Aspect ratio (w/h)");␣
 ↪axes[2].set_xlabel("ratio")
plt.tight_layout(); plt.show()
```

```
                  width                                height               \
                  count  min median        mean    max  count  min median
```

```
label
bengal                527  192  500.0  888.573055  6016    527  145  375.0
domestic_shorthair    520  190  300.0  881.950000  7793    520  148  400.0
maine_coon            540  116  500.0  841.179630  5616    540  155  500.0
ragdoll               557  172  500.0  932.976661  6000    557  112  456.0
siamese              1505  256  256.0  500.795349  6000   1505  256  256.0
```

```
                                aspect                                  \
                          mean    max  count       min    median      mean
label
bengal                743.022770  4928    527  0.496000  1.333333  1.247432
domestic_shorthair    865.788462  5472    520  0.483871  1.000000  1.042985
maine_coon            832.737037  6000    540  0.483871  1.000000  1.065800
ragdoll               861.725314  5389    557  0.476948  1.259214  1.160972
siamese               469.294352  4630   1505  0.495495  1.000000  1.026400
```

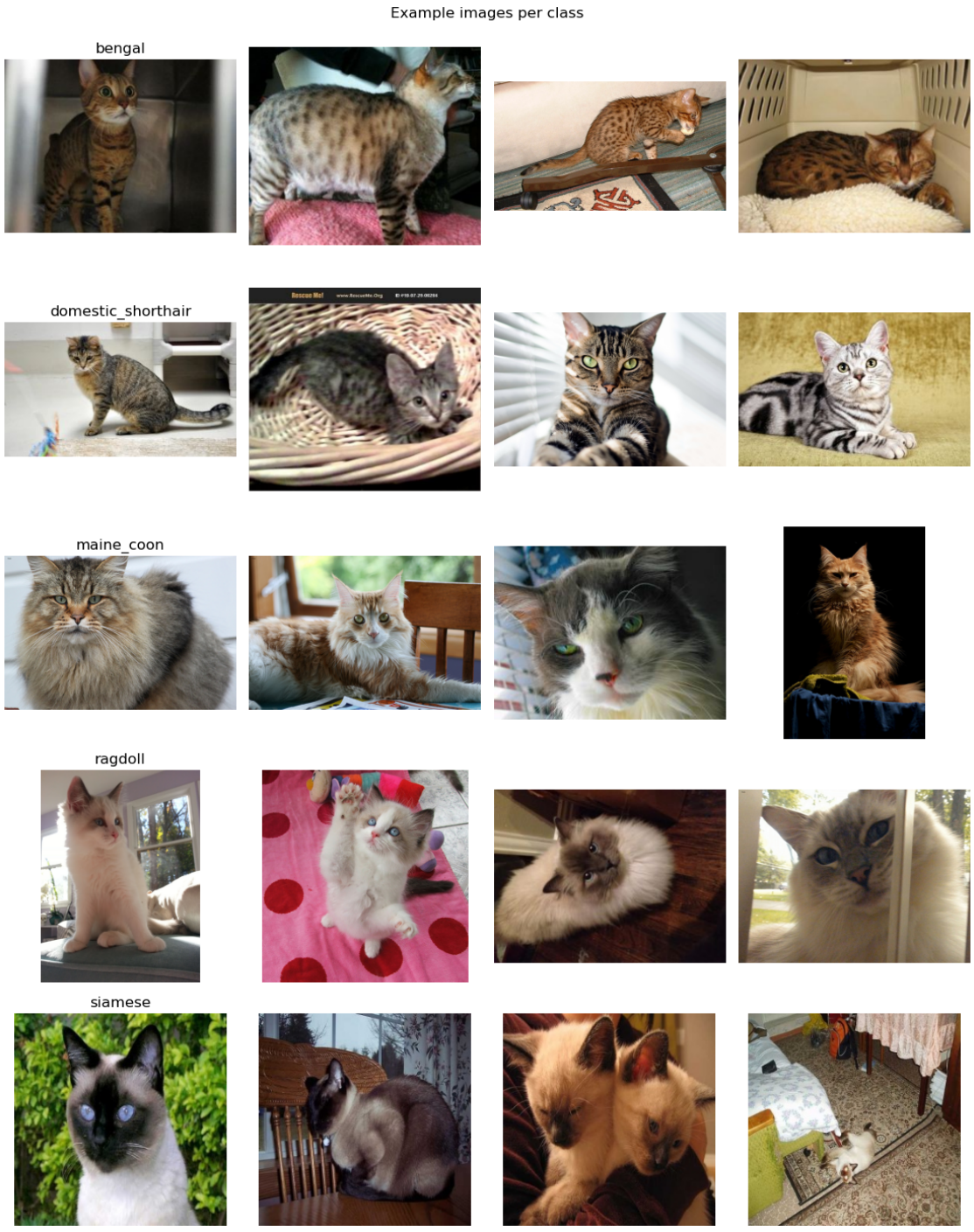
```
                          megapix
                              max  count       min    median      mean       max
label
bengal                   2.173913    527  0.031872  0.167000  1.368529  24.064000
domestic_shorthair       2.027027    520  0.044400  0.120000  1.684241  40.484635
maine_coon               2.183406    540  0.017980  0.185250  1.324973  25.000000
ragdoll                  2.678571    557  0.028552  0.187500  1.510091  24.000000
siamese                  2.370370   1505  0.065536  0.065536  0.671737  24.000000
```



Next, we show a small grid of example images for each class with their labels. This is a quick visual check that the data look right.

[6]:
```
# Examples & labels (image grid)
def show_examples(df, per_class=4):
    labels = sorted(df["label"].unique())
    nrows, ncols = len(labels), per_class
    fig, axes = plt.subplots(nrows, ncols, figsize=(2.8*ncols, 2.8*nrows))
    if nrows == 1: axes = np.array([axes])
    for r, lab in enumerate(labels):
        subset = df[df["label"]==lab]
```

```
        sample = subset.sample(min(per_class, len(subset)), random_state=42)
        for c, (_, row) in enumerate(sample.iterrows()):
            ax = axes[r, c] if ncols>1 else axes[r, 0]
            img = Image.open(row["path"]).convert("RGB")
            ax.imshow(img); ax.set_axis_off()
            if c == 0: ax.set_title(lab)
    plt.suptitle("Example images per class", y=0.995)
    plt.tight_layout(); plt.show()

show_examples(meta, per_class=4)
```

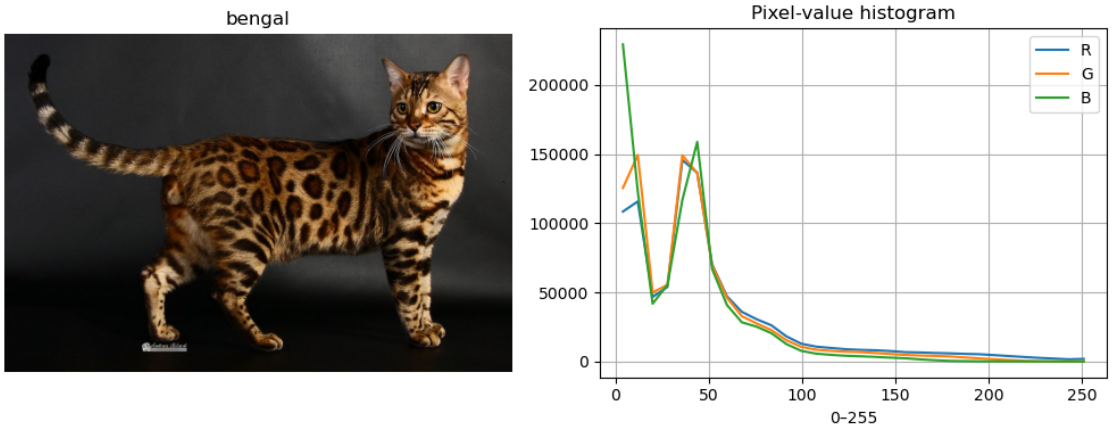


We also plot RGB histograms for one image per class. That gives a basic view of brightness and color for single images.

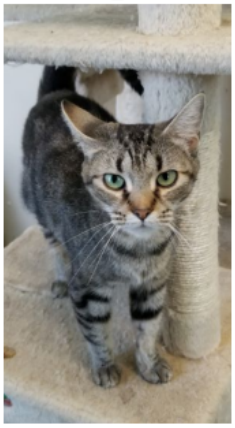
[7]:
```
# Histograms on individual images
def plot_image_and_histogram(image_path, bins=32):
```

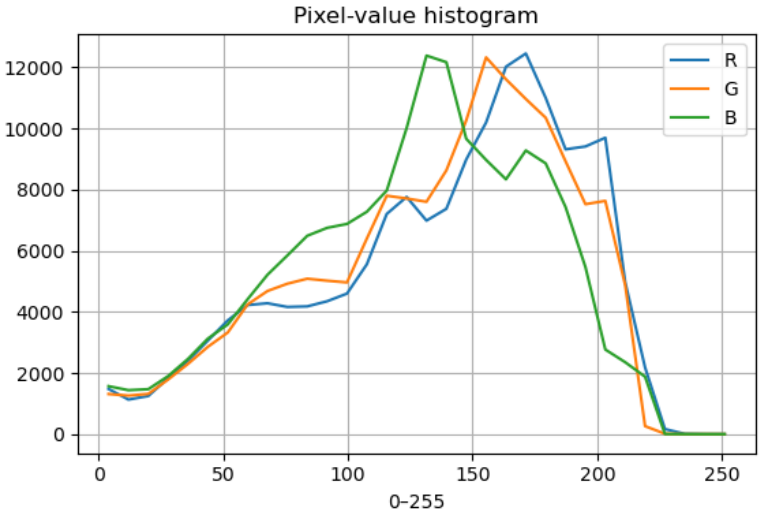
```
    im = Image.open(image_path).convert("RGB")
    arr = np.asarray(im, dtype=np.uint8)
    fig = plt.figure(figsize=(10,4))
    ax1 = plt.subplot(1,2,1)
    ax1.imshow(im); ax1.set_axis_off(); ax1.set_title(Path(image_path).parent.
 ↪name)
    ax2 = plt.subplot(1,2,2)
    for ch, name in enumerate(["R","G","B"]):
        hist, edges = np.histogram(arr[...,ch].ravel(), bins=bins,␣
 ↪range=(0,255))
        centers = 0.5*(edges[1:]+edges[:-1])
        ax2.plot(centers, hist, label=name)
    ax2.set_title("Pixel-value histogram"); ax2.set_xlabel("0-255"); ax2.
 ↪legend()
    plt.tight_layout(); plt.show()

print("Single-image histograms (one per class):")
for lab in sorted(meta["label"].unique()):
    ex = meta[meta["label"]==lab]
    if len(ex):
        plot_image_and_histogram(ex.iloc[0]["path"], bins=32)
```

```
Single-image histograms (one per class):
```

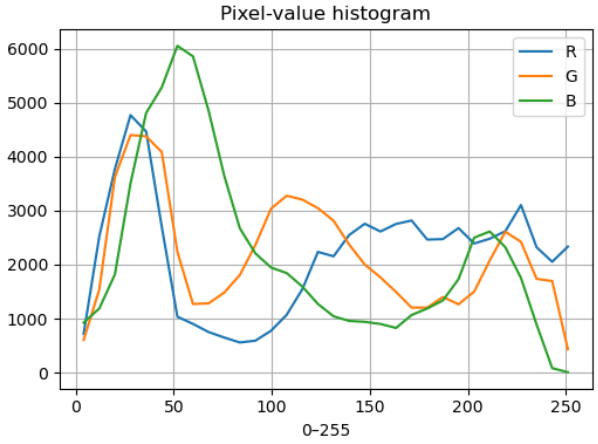


domestic_shorthair

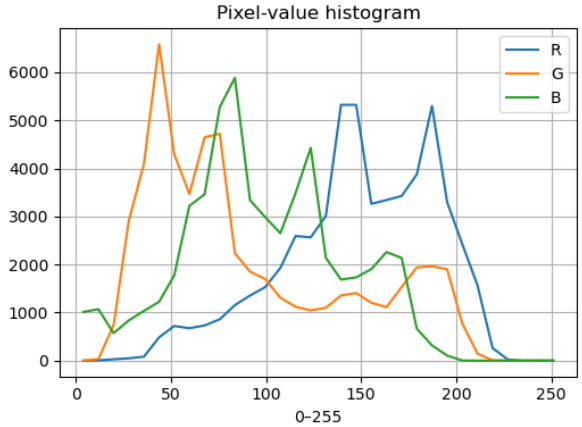


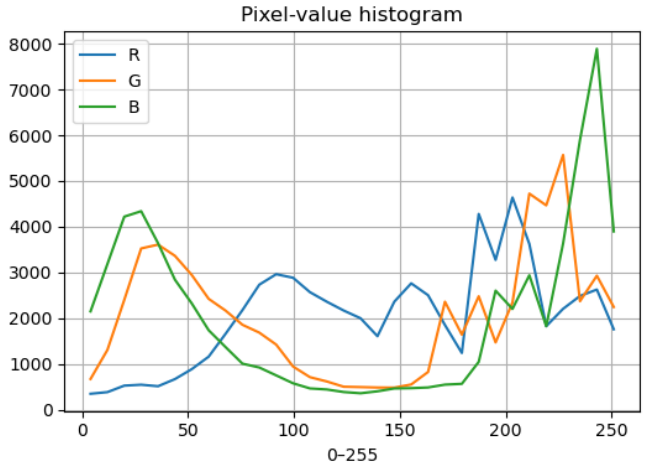


maine_coon



ragdoll





The above data analysis gives us a clear picture of what the dataset includes. We know the folder layout for loaders, the class balance, typical image sizes and aspect ratios, and the formats/color modes. Hopefully this will help us be more prepared for the later challenge of actually training a model.

[8]:
```
import shutil
from PIL import Image, ImageOps

def standardize_images(
    df,
    out_root="CatData/_tmp_standardized",
    target_size=(299,299),
    out_format="JPEG",
    quality=95,
    preserve_structure=True,
    letterbox=False,
    bg_color=(0,0,0),
    val_fraction=0.10,
    test_fraction=0.10,
    random_state=42,
    balance_to_min_class=False,    # <--- NEW
):
    """
    df: metadata dataframe with columns 'path','label'
    ...
    """

    assert 0 <= val_fraction < 1
    assert 0 <= test_fraction < 1
```

```
    assert val_fraction + test_fraction < 1, "val_fraction + test_fraction must␣
↪be < 1"

    df = df.copy()

    # --- NEW: balance to smallest class ---
    if balance_to_min_class:
        counts = df['label'].value_counts()
        min_count = counts.min()
        df = (
            df
            .groupby('label', group_keys=False)
            .apply(lambda g: g.sample(min_count, random_state=random_state))
            .reset_index(drop=True)
        )
        print(f"Balanced dataset: {len(df)} images "
              f"({min_count} per class, {df['label'].nunique()} classes)")

    # Clean output directory
    out_root = Path(out_root)
    if out_root.exists():
        shutil.rmtree(out_root)
    out_root.mkdir(parents=True, exist_ok=True)

    rng = np.random.default_rng(random_state)

    # Decide train/val/test assignment
    rand_vals = rng.random(len(df))

    df["split"] = "train"  # default
    df.loc[rand_vals < test_fraction, "split"] = "test"
    df.loc[
        (rand_vals >= test_fraction) & (rand_vals < test_fraction +␣
↪val_fraction),
        "split"
    ] = "val"

    counts = 0
    for _, row in df.iterrows():
        src = Path(row["path"])
        label = row["label"]
        split = row["split"]

        try:
            with Image.open(src) as im:
                im = im.convert("RGB")
```

```
                if letterbox:
                    im.thumbnail(target_size, Image.Resampling.LANCZOS)
                    canvas = Image.new("RGB", target_size, bg_color)
                    off_x = (target_size[0] - im.width) // 2
                    off_y = (target_size[1] - im.height) // 2
                    canvas.paste(im, (off_x, off_y))
                    out_im = canvas
                else:
                    out_im = ImageOps.fit(im, target_size, Image.Resampling.
 ↪LANCZOS)

                split_dir = out_root / split
                subdir = split_dir / label if preserve_structure else split_dir
                subdir.mkdir(parents=True, exist_ok=True)

                out_name = src.stem + "." + ("jpg" if out_format.
 ↪upper()=="JPEG" else out_format.lower())
                out_path = subdir / out_name

                save_kwargs = {}
                if out_format.upper() == "JPEG":
                    save_kwargs.update({"quality": quality, "optimize": True})

                out_im.save(out_path, out_format, **save_kwargs)
                counts += 1

        except Exception as e:
            print(f"Failed {src}: {e}")

    print(f"Standardized {counts} images -> {out_root}")
    print(f"Train size: {(df['split']=='train').sum()}")
    print(f"Val size:   {(df['split']=='val').sum()}")
    print(f"Test size:  {(df['split']=='test').sum()}")

    return df

meta_split = standardize_images(
    meta,
    target_size=(299, 299),
    out_format="JPEG",
    letterbox=False,
    val_fraction=0.10,
    test_fraction=0.10,
    balance_to_min_class=True,
)
```

```
/tmp/ipykernel_2642276/1203097361.py:36: DeprecationWarning:
```

```
DataFrameGroupBy.apply operated on the grouping columns. This behavior is
deprecated, and in a future version of pandas the grouping columns will be
excluded from the operation. Either pass `include_groups=False` to exclude the
groupings or explicitly select the grouping columns after groupby to silence
this warning.
  .apply(lambda g: g.sample(min_count, random_state=random_state))

Balanced dataset: 2600 images (520 per class, 5 classes)
Standardized 2600 images -> CatData/_tmp_standardized
Train size: 2060
Val size:   282
Test size:  258
```

5 Choice of ML algorithm

For this project we frame cat-breed recognition as a supervised multiclass image-classification problem as each input image is paired with a known breed label, and the model must assign one of five possible classes (Bengal, Domestic Shorthair, Maine Coon, Ragdoll, Siamese). Because the input data are images with a clear two-dimensional spatial structure, we use computer-vision methods rather than generic tabular algorithms. Convolutional neural networks (CNNs) are the standard choice for image recognition on large datasets such as ImageNet.

CNNs are well suited to this task because they exploit the topology of image data through local receptive fields and weight sharing. This allows the network to learn translation-invariant features, so if the cat is shifted, rotated, or appears at a different scale, the same convolutional filters can still detect relevant visual patterns such as edges, fur textures, and facial structures. Instead of relying on hand-crafted features like manually defined color histograms or shape descriptors, the CNN automatically learns a hierarchy of representations, from low-level edges and corners in the early layers to higher-level concepts such as ears, eyes, and overall head shape in deeper layers. This makes CNNs more robust to variations in pose, lighting, and background that characterize real-world cat photos.

Training a deep CNN from scratch usually requires very large labeled datasets and substantial computational resources. Our cat-breed dataset is much smaller (952) than ImageNet (14 million), so directly learning all weights from random initialization would increase the risk of overfitting and lengthen training time. To address this, we adopt transfer learning, which is standard practice for image classification when data are limited. We reuse a pre-trained CNN, Inception v3, which is originally trained on more than one million images and 1,000 object categories from the ImageNet dataset. Inception v3 is a deep, optimized CNN architecture that combines convolutional, pooling, and "Inception" modules to achieve strong accuracy without using too much computational powers.

In our transfer-learning setup, the lower and middle layers of the pre-trained Inception v3 model are frozen, because they capture generic visual features such as edges, textures, and simple shapes that are useful for almost any image domain. Only the final classification layers are replaced and retrained on our five cat-breed classes. This approach drastically reduces training time and data requirements while typically improving generalization compared with training a similar-sized network from scratch on a small dataset.

We implement our model in Python using TensorFlow and its high-level Keras API. Keras provides a simple way to define, train, and evaluate deep learning models, while TensorFlow handles the

underlying computation efficiently on both CPU and GPU. This framework also offers built-in support for loading pre-trained models such as Inception v3 and makes it straightforward to fine-tune them for our specific cat-breed classification task.

5.1 Sources

- Szegedy, C. et al. "Rethinking the Inception Architecture for Computer Vision." CVPR, 2016.
- ImageNet Project. "About ImageNet."
- Kaggle. "Inception V3 Model"
- Tensorflow Learn. "Convolutional Neural Network (CNN)"
- Wikipedia. "Convolutional Neural Network."
- Wikipedia. "Inception (deep learning architecture)."
- Keras: Transfer learning & fine-tuning"

6 ML Data processing

her

7 Performance metrics

Here

8 Under- and overfitting

here

9 Optimizations and improvements

9.1 Confusion matrix

10 Conclusion

Here

11 KODE

[9]:
```
import tensorflow as tf
from tensorflow.keras.applications import InceptionV3
from tensorflow.keras import layers, models
```

```
NUM_CAT_BREEDS = 5
width, height = 299, 299

base_model = InceptionV3(
    weights="imagenet",
    include_top=False,
    input_shape=(width,height,3)
)
```

```
2025-11-27 16:08:47.726404: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.727691: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.728906: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.730078: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.731251: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.732431: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.743134: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
```

```
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.744336: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.745545: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.746753: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.747950: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.749101: I
tensorflow/core/common_runtime/gpu/gpu_device.cc:1929] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 10272 MB memory:  -> device:
0, name: NVIDIA GeForce RTX 3060, pci bus id: 0000:0a:00.0, compute capability:
8.6
2025-11-27 16:08:47.749454: I
external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful
NUMA node read from SysFS had negative value (-1), but there must be at least
one NUMA node, so returning NUMA node zero. See more at
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-
pci#L344-L355
2025-11-27 16:08:47.750584: I
tensorflow/core/common_runtime/gpu/gpu_device.cc:1929] Created device
/job:localhost/replica:0/task:0/device:GPU:1 with 10273 MB memory:  -> device:
1, name: NVIDIA GeForce RTX 3060, pci bus id: 0000:0b:00.0, compute capability:
8.6
```

[10]:
```
base_model.trainable = False
```

[11]:
```
inputs = tf.keras.Input(shape=(width, height, 3))
```

[12]:
```
from tensorflow.keras.applications.inception_v3 import preprocess_input
```

[13]:
```
x = preprocess_input(inputs)

x = base_model(x, training=False)

x = layers.GlobalAveragePooling2D()(x)

x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.5)(x)

outputs = layers.Dense(NUM_CAT_BREEDS, activation="softmax")(x)

model = models.Model(inputs, outputs)
```

[14]:
```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=['accuracy']
)
```

[15]:
```
IMG_SIZE = (width, height)
BATCH_SIZE = 32

train_ds = tf.keras.utils.image_dataset_from_directory(
    "CatData/_tmp_standardized/train",
    validation_split=.2,
    subset="training",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    "CatData/_tmp_standardized/val",
    validation_split=.2,
    subset="training",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

test_ds = tf.keras.utils.image_dataset_from_directory(
    "CatData/_tmp_standardized/test",
    shuffle=True,
```

```
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

class_names = train_ds.class_names
NUM_CLASSES = len(class_names)
print(class_names)
```

```
Found 2060 files belonging to 5 classes.
Using 1648 files for training.
Found 282 files belonging to 5 classes.
Using 226 files for training.
Found 258 files belonging to 5 classes.
['bengal', 'domestic_shorthair', 'maine_coon', 'ragdoll', 'siamese']
```

[16]:
```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(.1),
    tf.keras.layers.RandomZoom(.1)
])
```

[17]:
```
# Include augmentation in the model
x = data_augmentation(inputs)
x = base_model(x, training=False)
```

[18]:
```
# Train the head
import numpy as np
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=300,
    restore_best_weights=True
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=500,
    callbacks=[early_stop]
)

print("Number of epochs run:", len(history.history["loss"]))
print("Best epoch according to callback:", early_stop.best_epoch)
print("Argmin of val_loss:", np.argmin(history.history["val_loss"]))
```

```
Epoch 1/500
```

```
2025-11-27 16:08:52.657097: I
external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:454] Loaded cuDNN
version 8800
2025-11-27 16:08:56.809554: I external/local_xla/xla/service/service.cc:168] XLA
service 0x55e321658950 initialized for platform CUDA (this does not guarantee
that XLA will be used). Devices:
2025-11-27 16:08:56.809584: I external/local_xla/xla/service/service.cc:176]
StreamExecutor device (0): NVIDIA GeForce RTX 3060, Compute Capability 8.6
2025-11-27 16:08:56.809588: I external/local_xla/xla/service/service.cc:176]
StreamExecutor device (1): NVIDIA GeForce RTX 3060, Compute Capability 8.6
2025-11-27 16:08:56.813317: I
tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269] disabling MLIR
crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRECTORY` to enable.
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1764256136.870594 2642393 device_compiler.h:186] Compiled cluster
using XLA!  This line is logged at most once for the lifetime of the process.

52/52 [==============================] - 16s 191ms/step - loss: 0.7477 -
accuracy: 0.6978 - val_loss: 0.4481 - val_accuracy: 0.8451
Epoch 2/500
52/52 [==============================] - 5s 92ms/step - loss: 0.4661 - accuracy:
0.8331 - val_loss: 0.3965 - val_accuracy: 0.8540
Epoch 3/500
52/52 [==============================] - 5s 92ms/step - loss: 0.4011 - accuracy:
0.8532 - val_loss: 0.3750 - val_accuracy: 0.8717
Epoch 4/500
52/52 [==============================] - 5s 92ms/step - loss: 0.3930 - accuracy:
0.8604 - val_loss: 0.3674 - val_accuracy: 0.8584
Epoch 5/500
52/52 [==============================] - 5s 92ms/step - loss: 0.3148 - accuracy:
0.8962 - val_loss: 0.3542 - val_accuracy: 0.8805
Epoch 6/500
52/52 [==============================] - 5s 92ms/step - loss: 0.3041 - accuracy:
0.8841 - val_loss: 0.3427 - val_accuracy: 0.8761
Epoch 7/500
52/52 [==============================] - 5s 91ms/step - loss: 0.2998 - accuracy:
0.8993 - val_loss: 0.3554 - val_accuracy: 0.8761
Epoch 8/500
52/52 [==============================] - 5s 91ms/step - loss: 0.2600 - accuracy:
0.9096 - val_loss: 0.3570 - val_accuracy: 0.8850
Epoch 9/500
52/52 [==============================] - 5s 91ms/step - loss: 0.2503 - accuracy:
0.9181 - val_loss: 0.3501 - val_accuracy: 0.8761
Epoch 10/500
52/52 [==============================] - 5s 91ms/step - loss: 0.2303 - accuracy:
0.9205 - val_loss: 0.4134 - val_accuracy: 0.8673
Epoch 11/500
```

```
52/52 [==============================] - 5s 92ms/step - loss: 0.2154 - accuracy:
0.9242 - val_loss: 0.3615 - val_accuracy: 0.8850
Epoch 12/500
52/52 [==============================] - 5s 91ms/step - loss: 0.2004 - accuracy:
0.9302 - val_loss: 0.3619 - val_accuracy: 0.8761
Epoch 13/500
52/52 [==============================] - 5s 90ms/step - loss: 0.1919 - accuracy:
0.9284 - val_loss: 0.3624 - val_accuracy: 0.8850
Epoch 14/500
52/52 [==============================] - 5s 89ms/step - loss: 0.2036 - accuracy:
0.9302 - val_loss: 0.3751 - val_accuracy: 0.8850
Epoch 15/500
52/52 [==============================] - 5s 91ms/step - loss: 0.1686 - accuracy:
0.9424 - val_loss: 0.3402 - val_accuracy: 0.8982
Epoch 16/500
52/52 [==============================] - 5s 91ms/step - loss: 0.1885 - accuracy:
0.9326 - val_loss: 0.3780 - val_accuracy: 0.8717
Epoch 17/500
52/52 [==============================] - 5s 92ms/step - loss: 0.1770 - accuracy:
0.9405 - val_loss: 0.3741 - val_accuracy: 0.8850
Epoch 18/500
52/52 [==============================] - 5s 92ms/step - loss: 0.1561 - accuracy:
0.9430 - val_loss: 0.3803 - val_accuracy: 0.8761
Epoch 19/500
52/52 [==============================] - 5s 92ms/step - loss: 0.1243 - accuracy:
0.9618 - val_loss: 0.4364 - val_accuracy: 0.8717
Epoch 20/500
52/52 [==============================] - 5s 91ms/step - loss: 0.1281 - accuracy:
0.9575 - val_loss: 0.3880 - val_accuracy: 0.8805
Epoch 21/500
52/52 [==============================] - 5s 92ms/step - loss: 0.1225 - accuracy:
0.9539 - val_loss: 0.3822 - val_accuracy: 0.8894
Epoch 22/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0983 - accuracy:
0.9697 - val_loss: 0.3890 - val_accuracy: 0.8850
Epoch 23/500
52/52 [==============================] - 5s 92ms/step - loss: 0.1163 - accuracy:
0.9612 - val_loss: 0.3965 - val_accuracy: 0.8805
Epoch 24/500
52/52 [==============================] - 5s 92ms/step - loss: 0.1215 - accuracy:
0.9630 - val_loss: 0.4098 - val_accuracy: 0.8628
Epoch 25/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0963 - accuracy:
0.9684 - val_loss: 0.4016 - val_accuracy: 0.8805
Epoch 26/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0839 - accuracy:
0.9721 - val_loss: 0.3907 - val_accuracy: 0.8938
Epoch 27/500
```

```
52/52 [==============================] - 5s 92ms/step - loss: 0.0971 - accuracy:
0.9678 - val_loss: 0.3894 - val_accuracy: 0.8673
Epoch 28/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0731 - accuracy:
0.9794 - val_loss: 0.4446 - val_accuracy: 0.8805
Epoch 29/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0762 - accuracy:
0.9727 - val_loss: 0.4355 - val_accuracy: 0.8717
Epoch 30/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0617 - accuracy:
0.9842 - val_loss: 0.4690 - val_accuracy: 0.8717
Epoch 31/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0790 - accuracy:
0.9727 - val_loss: 0.4493 - val_accuracy: 0.8584
Epoch 32/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0583 - accuracy:
0.9806 - val_loss: 0.4145 - val_accuracy: 0.8805
Epoch 33/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0551 - accuracy:
0.9854 - val_loss: 0.4313 - val_accuracy: 0.8805
Epoch 34/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0523 - accuracy:
0.9836 - val_loss: 0.5298 - val_accuracy: 0.8761
Epoch 35/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0519 - accuracy:
0.9854 - val_loss: 0.4542 - val_accuracy: 0.8850
Epoch 36/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0490 - accuracy:
0.9873 - val_loss: 0.4622 - val_accuracy: 0.8894
Epoch 37/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0612 - accuracy:
0.9794 - val_loss: 0.4647 - val_accuracy: 0.9027
Epoch 38/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0537 - accuracy:
0.9806 - val_loss: 0.4826 - val_accuracy: 0.8805
Epoch 39/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0438 - accuracy:
0.9873 - val_loss: 0.4739 - val_accuracy: 0.8894
Epoch 40/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0477 - accuracy:
0.9848 - val_loss: 0.5132 - val_accuracy: 0.8850
Epoch 41/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0487 - accuracy:
0.9854 - val_loss: 0.4795 - val_accuracy: 0.8894
Epoch 42/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0457 - accuracy:
0.9879 - val_loss: 0.4955 - val_accuracy: 0.8894
Epoch 43/500
```

```
52/52 [==============================] - 5s 92ms/step - loss: 0.0507 - accuracy:
0.9860 - val_loss: 0.4738 - val_accuracy: 0.8761
Epoch 44/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0445 - accuracy:
0.9879 - val_loss: 0.5060 - val_accuracy: 0.8673
Epoch 45/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0408 - accuracy:
0.9879 - val_loss: 0.5206 - val_accuracy: 0.8805
Epoch 46/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0531 - accuracy:
0.9824 - val_loss: 0.5433 - val_accuracy: 0.8761
Epoch 47/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0490 - accuracy:
0.9848 - val_loss: 0.5055 - val_accuracy: 0.8894
Epoch 48/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0505 - accuracy:
0.9812 - val_loss: 0.5068 - val_accuracy: 0.8938
Epoch 49/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0569 - accuracy:
0.9818 - val_loss: 0.5106 - val_accuracy: 0.8938
Epoch 50/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0481 - accuracy:
0.9812 - val_loss: 0.5318 - val_accuracy: 0.8761
Epoch 51/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0474 - accuracy:
0.9848 - val_loss: 0.5978 - val_accuracy: 0.8717
Epoch 52/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0512 - accuracy:
0.9830 - val_loss: 0.6603 - val_accuracy: 0.8540
Epoch 53/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0526 - accuracy:
0.9788 - val_loss: 0.5523 - val_accuracy: 0.8584
Epoch 54/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0515 - accuracy:
0.9794 - val_loss: 0.5798 - val_accuracy: 0.8805
Epoch 55/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0427 - accuracy:
0.9842 - val_loss: 0.5951 - val_accuracy: 0.8451
Epoch 56/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0316 - accuracy:
0.9891 - val_loss: 0.5330 - val_accuracy: 0.9027
Epoch 57/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0293 - accuracy:
0.9909 - val_loss: 0.5684 - val_accuracy: 0.8805
Epoch 58/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0288 - accuracy:
0.9909 - val_loss: 0.5546 - val_accuracy: 0.8938
Epoch 59/500
```

```
52/52 [==============================] - 5s 92ms/step - loss: 0.0377 - accuracy:
0.9879 - val_loss: 0.6074 - val_accuracy: 0.8938
Epoch 60/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0879 - accuracy:
0.9697 - val_loss: 0.5507 - val_accuracy: 0.8982
Epoch 61/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0706 - accuracy:
0.9715 - val_loss: 0.5673 - val_accuracy: 0.8717
Epoch 62/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0663 - accuracy:
0.9769 - val_loss: 0.5185 - val_accuracy: 0.8717
Epoch 63/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0581 - accuracy:
0.9788 - val_loss: 0.5499 - val_accuracy: 0.8761
Epoch 64/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0589 - accuracy:
0.9751 - val_loss: 0.5664 - val_accuracy: 0.8761
Epoch 65/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0459 - accuracy:
0.9842 - val_loss: 0.5533 - val_accuracy: 0.8805
Epoch 66/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0295 - accuracy:
0.9921 - val_loss: 0.5859 - val_accuracy: 0.8850
Epoch 67/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0222 - accuracy:
0.9939 - val_loss: 0.6092 - val_accuracy: 0.8850
Epoch 68/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0342 - accuracy:
0.9879 - val_loss: 0.5482 - val_accuracy: 0.8894
Epoch 69/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0388 - accuracy:
0.9842 - val_loss: 0.5585 - val_accuracy: 0.8717
Epoch 70/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0378 - accuracy:
0.9879 - val_loss: 0.5822 - val_accuracy: 0.8805
Epoch 71/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0398 - accuracy:
0.9873 - val_loss: 0.5852 - val_accuracy: 0.8761
Epoch 72/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0356 - accuracy:
0.9891 - val_loss: 0.5667 - val_accuracy: 0.8894
Epoch 73/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0764 - accuracy:
0.9739 - val_loss: 0.5854 - val_accuracy: 0.8894
Epoch 74/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0538 - accuracy:
0.9830 - val_loss: 0.5563 - val_accuracy: 0.8805
Epoch 75/500
```

```
52/52 [==============================] - 5s 91ms/step - loss: 0.0471 - accuracy:
0.9836 - val_loss: 0.5545 - val_accuracy: 0.8938
Epoch 76/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0367 - accuracy:
0.9891 - val_loss: 0.5080 - val_accuracy: 0.8982
Epoch 77/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0328 - accuracy:
0.9909 - val_loss: 0.5552 - val_accuracy: 0.8938
Epoch 78/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0368 - accuracy:
0.9879 - val_loss: 0.5795 - val_accuracy: 0.8850
Epoch 79/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0330 - accuracy:
0.9873 - val_loss: 0.6755 - val_accuracy: 0.8805
Epoch 80/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0364 - accuracy:
0.9867 - val_loss: 0.7026 - val_accuracy: 0.8894
Epoch 81/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0472 - accuracy:
0.9848 - val_loss: 0.5826 - val_accuracy: 0.8894
Epoch 82/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0536 - accuracy:
0.9788 - val_loss: 0.7002 - val_accuracy: 0.8894
Epoch 83/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0558 - accuracy:
0.9782 - val_loss: 0.5660 - val_accuracy: 0.8717
Epoch 84/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0362 - accuracy:
0.9891 - val_loss: 0.6542 - val_accuracy: 0.8761
Epoch 85/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0327 - accuracy:
0.9891 - val_loss: 0.5748 - val_accuracy: 0.8894
Epoch 86/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0479 - accuracy:
0.9812 - val_loss: 0.5921 - val_accuracy: 0.8938
Epoch 87/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0306 - accuracy:
0.9897 - val_loss: 0.6062 - val_accuracy: 0.8805
Epoch 88/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0338 - accuracy:
0.9897 - val_loss: 0.6413 - val_accuracy: 0.8850
Epoch 89/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0286 - accuracy:
0.9903 - val_loss: 0.6816 - val_accuracy: 0.8761
Epoch 90/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0249 - accuracy:
0.9927 - val_loss: 0.6535 - val_accuracy: 0.8894
Epoch 91/500
```

```
52/52 [==============================] - 5s 91ms/step - loss: 0.0321 - accuracy:
0.9873 - val_loss: 0.6086 - val_accuracy: 0.8850
Epoch 92/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0530 - accuracy:
0.9800 - val_loss: 0.6113 - val_accuracy: 0.8717
Epoch 93/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0430 - accuracy:
0.9848 - val_loss: 0.5844 - val_accuracy: 0.8894
Epoch 94/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0542 - accuracy:
0.9788 - val_loss: 0.5534 - val_accuracy: 0.8894
Epoch 95/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0554 - accuracy:
0.9812 - val_loss: 0.5670 - val_accuracy: 0.8938
Epoch 96/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0374 - accuracy:
0.9860 - val_loss: 0.6107 - val_accuracy: 0.8894
Epoch 97/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0258 - accuracy:
0.9909 - val_loss: 0.6547 - val_accuracy: 0.8982
Epoch 98/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0301 - accuracy:
0.9903 - val_loss: 0.6235 - val_accuracy: 0.8894
Epoch 99/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0387 - accuracy:
0.9854 - val_loss: 0.6109 - val_accuracy: 0.8805
Epoch 100/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0291 - accuracy:
0.9897 - val_loss: 0.8072 - val_accuracy: 0.8717
Epoch 101/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0402 - accuracy:
0.9854 - val_loss: 0.7012 - val_accuracy: 0.8894
Epoch 102/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0287 - accuracy:
0.9885 - val_loss: 0.6553 - val_accuracy: 0.8761
Epoch 103/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0290 - accuracy:
0.9885 - val_loss: 0.7022 - val_accuracy: 0.8717
Epoch 104/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0518 - accuracy:
0.9854 - val_loss: 0.7340 - val_accuracy: 0.8761
Epoch 105/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0444 - accuracy:
0.9848 - val_loss: 0.7189 - val_accuracy: 0.8628
Epoch 106/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0319 - accuracy:
0.9891 - val_loss: 0.6708 - val_accuracy: 0.8894
Epoch 107/500
```

```
52/52 [==============================] - 5s 92ms/step - loss: 0.0273 - accuracy:
0.9915 - val_loss: 0.6177 - val_accuracy: 0.8761
Epoch 108/500
52/52 [==============================] - 5s 95ms/step - loss: 0.0183 - accuracy:
0.9945 - val_loss: 0.6976 - val_accuracy: 0.8850
Epoch 109/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0410 - accuracy:
0.9830 - val_loss: 0.7282 - val_accuracy: 0.8805
Epoch 110/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0209 - accuracy:
0.9891 - val_loss: 0.7219 - val_accuracy: 0.8894
Epoch 111/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0226 - accuracy:
0.9933 - val_loss: 0.7347 - val_accuracy: 0.8850
Epoch 112/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0203 - accuracy:
0.9939 - val_loss: 0.8416 - val_accuracy: 0.8805
Epoch 113/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0186 - accuracy:
0.9915 - val_loss: 0.8196 - val_accuracy: 0.8540
Epoch 114/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0222 - accuracy:
0.9897 - val_loss: 0.9563 - val_accuracy: 0.8805
Epoch 115/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0426 - accuracy:
0.9842 - val_loss: 0.7789 - val_accuracy: 0.8894
Epoch 116/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0286 - accuracy:
0.9921 - val_loss: 0.6669 - val_accuracy: 0.9027
Epoch 117/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0277 - accuracy:
0.9897 - val_loss: 0.8216 - val_accuracy: 0.8628
Epoch 118/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0193 - accuracy:
0.9958 - val_loss: 0.7800 - val_accuracy: 0.8894
Epoch 119/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0173 - accuracy:
0.9958 - val_loss: 0.7845 - val_accuracy: 0.8894
Epoch 120/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0154 - accuracy:
0.9939 - val_loss: 0.7922 - val_accuracy: 0.8805
Epoch 121/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0162 - accuracy:
0.9964 - val_loss: 0.7664 - val_accuracy: 0.8805
Epoch 122/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0131 - accuracy:
0.9958 - val_loss: 0.8189 - val_accuracy: 0.8894
Epoch 123/500
```

```
52/52 [==============================] - 5s 91ms/step - loss: 0.0182 - accuracy:
0.9945 - val_loss: 0.8057 - val_accuracy: 0.8761
Epoch 124/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0323 - accuracy:
0.9873 - val_loss: 0.7314 - val_accuracy: 0.8938
Epoch 125/500
52/52 [==============================] - 5s 95ms/step - loss: 0.0577 - accuracy:
0.9775 - val_loss: 0.7905 - val_accuracy: 0.8584
Epoch 126/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0604 - accuracy:
0.9751 - val_loss: 0.6939 - val_accuracy: 0.8894
Epoch 127/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0421 - accuracy:
0.9879 - val_loss: 0.6490 - val_accuracy: 0.8850
Epoch 128/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0337 - accuracy:
0.9879 - val_loss: 0.6450 - val_accuracy: 0.8805
Epoch 129/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0330 - accuracy:
0.9903 - val_loss: 0.6966 - val_accuracy: 0.8805
Epoch 130/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0495 - accuracy:
0.9818 - val_loss: 0.8285 - val_accuracy: 0.8761
Epoch 131/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0309 - accuracy:
0.9879 - val_loss: 0.6520 - val_accuracy: 0.8805
Epoch 132/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0194 - accuracy:
0.9921 - val_loss: 0.7755 - val_accuracy: 0.8850
Epoch 133/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0218 - accuracy:
0.9945 - val_loss: 0.7688 - val_accuracy: 0.8805
Epoch 134/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0174 - accuracy:
0.9933 - val_loss: 0.7403 - val_accuracy: 0.9027
Epoch 135/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0187 - accuracy:
0.9939 - val_loss: 0.8407 - val_accuracy: 0.8850
Epoch 136/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0272 - accuracy:
0.9921 - val_loss: 0.9010 - val_accuracy: 0.8761
Epoch 137/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0412 - accuracy:
0.9854 - val_loss: 0.7835 - val_accuracy: 0.8850
Epoch 138/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0337 - accuracy:
0.9879 - val_loss: 0.7061 - val_accuracy: 0.8850
Epoch 139/500
```

```
52/52 [==============================] - 5s 89ms/step - loss: 0.0480 - accuracy:
0.9824 - val_loss: 0.7605 - val_accuracy: 0.8850
Epoch 140/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0290 - accuracy:
0.9885 - val_loss: 0.7211 - val_accuracy: 0.8717
Epoch 141/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0258 - accuracy:
0.9897 - val_loss: 0.9136 - val_accuracy: 0.8496
Epoch 142/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0345 - accuracy:
0.9879 - val_loss: 0.8816 - val_accuracy: 0.8673
Epoch 143/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0551 - accuracy:
0.9769 - val_loss: 0.6181 - val_accuracy: 0.8894
Epoch 144/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0416 - accuracy:
0.9836 - val_loss: 0.7583 - val_accuracy: 0.8717
Epoch 145/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0458 - accuracy:
0.9836 - val_loss: 0.8016 - val_accuracy: 0.8805
Epoch 146/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0406 - accuracy:
0.9860 - val_loss: 0.8331 - val_accuracy: 0.8894
Epoch 147/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0383 - accuracy:
0.9854 - val_loss: 0.7431 - val_accuracy: 0.8761
Epoch 148/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0519 - accuracy:
0.9848 - val_loss: 0.7405 - val_accuracy: 0.8850
Epoch 149/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0291 - accuracy:
0.9915 - val_loss: 0.7506 - val_accuracy: 0.8850
Epoch 150/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0279 - accuracy:
0.9903 - val_loss: 0.9147 - val_accuracy: 0.8761
Epoch 151/500
52/52 [==============================] - 5s 95ms/step - loss: 0.0274 - accuracy:
0.9897 - val_loss: 0.8013 - val_accuracy: 0.8850
Epoch 152/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0216 - accuracy:
0.9909 - val_loss: 0.8834 - val_accuracy: 0.8938
Epoch 153/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0231 - accuracy:
0.9921 - val_loss: 0.7819 - val_accuracy: 0.8850
Epoch 154/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0139 - accuracy:
0.9951 - val_loss: 1.0007 - val_accuracy: 0.8894
Epoch 155/500
```

```
52/52 [==============================] - 5s 90ms/step - loss: 0.0178 - accuracy:
0.9927 - val_loss: 0.8798 - val_accuracy: 0.8894
Epoch 156/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0194 - accuracy:
0.9939 - val_loss: 0.8552 - val_accuracy: 0.8894
Epoch 157/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0178 - accuracy:
0.9939 - val_loss: 0.8469 - val_accuracy: 0.8850
Epoch 158/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0203 - accuracy:
0.9939 - val_loss: 0.7966 - val_accuracy: 0.8850
Epoch 159/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0177 - accuracy:
0.9958 - val_loss: 0.8433 - val_accuracy: 0.8850
Epoch 160/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0188 - accuracy:
0.9933 - val_loss: 0.8555 - val_accuracy: 0.8717
Epoch 161/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0277 - accuracy:
0.9897 - val_loss: 0.7470 - val_accuracy: 0.8673
Epoch 162/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0377 - accuracy:
0.9891 - val_loss: 0.7819 - val_accuracy: 0.8717
Epoch 163/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0401 - accuracy:
0.9867 - val_loss: 0.7447 - val_accuracy: 0.8496
Epoch 164/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0440 - accuracy:
0.9836 - val_loss: 0.6883 - val_accuracy: 0.8894
Epoch 165/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0253 - accuracy:
0.9915 - val_loss: 0.8326 - val_accuracy: 0.8938
Epoch 166/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0265 - accuracy:
0.9885 - val_loss: 0.9345 - val_accuracy: 0.8850
Epoch 167/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0268 - accuracy:
0.9879 - val_loss: 0.7812 - val_accuracy: 0.8805
Epoch 168/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0220 - accuracy:
0.9933 - val_loss: 0.8668 - val_accuracy: 0.8894
Epoch 169/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0166 - accuracy:
0.9939 - val_loss: 0.8175 - val_accuracy: 0.9115
Epoch 170/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0175 - accuracy:
0.9939 - val_loss: 0.8806 - val_accuracy: 0.8805
Epoch 171/500
```

```
52/52 [==============================] - 5s 91ms/step - loss: 0.0199 - accuracy:
0.9945 - val_loss: 0.8895 - val_accuracy: 0.8894
Epoch 172/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0270 - accuracy:
0.9903 - val_loss: 0.9099 - val_accuracy: 0.8717
Epoch 173/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0339 - accuracy:
0.9879 - val_loss: 0.8715 - val_accuracy: 0.8805
Epoch 174/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0556 - accuracy:
0.9806 - val_loss: 0.7411 - val_accuracy: 0.8982
Epoch 175/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0599 - accuracy:
0.9788 - val_loss: 0.8215 - val_accuracy: 0.8850
Epoch 176/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0681 - accuracy:
0.9721 - val_loss: 0.7188 - val_accuracy: 0.8628
Epoch 177/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0371 - accuracy:
0.9885 - val_loss: 0.7541 - val_accuracy: 0.8805
Epoch 178/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0359 - accuracy:
0.9897 - val_loss: 0.8378 - val_accuracy: 0.8805
Epoch 179/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0288 - accuracy:
0.9891 - val_loss: 0.9699 - val_accuracy: 0.8938
Epoch 180/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0493 - accuracy:
0.9842 - val_loss: 0.8123 - val_accuracy: 0.8584
Epoch 181/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0203 - accuracy:
0.9945 - val_loss: 0.8514 - val_accuracy: 0.8894
Epoch 182/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0211 - accuracy:
0.9897 - val_loss: 0.8558 - val_accuracy: 0.8805
Epoch 183/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0327 - accuracy:
0.9885 - val_loss: 0.9092 - val_accuracy: 0.8938
Epoch 184/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0162 - accuracy:
0.9933 - val_loss: 0.9706 - val_accuracy: 0.8894
Epoch 185/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0146 - accuracy:
0.9939 - val_loss: 0.9518 - val_accuracy: 0.8761
Epoch 186/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0176 - accuracy:
0.9939 - val_loss: 0.9039 - val_accuracy: 0.8938
Epoch 187/500
```

```
52/52 [==============================] - 5s 90ms/step - loss: 0.0163 - accuracy:
0.9939 - val_loss: 0.8892 - val_accuracy: 0.8673
Epoch 188/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0138 - accuracy:
0.9939 - val_loss: 0.9209 - val_accuracy: 0.8805
Epoch 189/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0266 - accuracy:
0.9879 - val_loss: 0.9133 - val_accuracy: 0.8805
Epoch 190/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0451 - accuracy:
0.9830 - val_loss: 0.8618 - val_accuracy: 0.8761
Epoch 191/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0523 - accuracy:
0.9836 - val_loss: 0.8077 - val_accuracy: 0.8805
Epoch 192/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0463 - accuracy:
0.9854 - val_loss: 0.9258 - val_accuracy: 0.8673
Epoch 193/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0389 - accuracy:
0.9836 - val_loss: 0.8923 - val_accuracy: 0.8717
Epoch 194/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0390 - accuracy:
0.9818 - val_loss: 0.9268 - val_accuracy: 0.8673
Epoch 195/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0354 - accuracy:
0.9891 - val_loss: 0.9428 - val_accuracy: 0.8717
Epoch 196/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0337 - accuracy:
0.9873 - val_loss: 0.8339 - val_accuracy: 0.8850
Epoch 197/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0372 - accuracy:
0.9867 - val_loss: 0.8864 - val_accuracy: 0.8894
Epoch 198/500
52/52 [==============================] - 5s 94ms/step - loss: 0.0226 - accuracy:
0.9897 - val_loss: 0.8770 - val_accuracy: 0.8584
Epoch 199/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0320 - accuracy:
0.9854 - val_loss: 0.8388 - val_accuracy: 0.8761
Epoch 200/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0446 - accuracy:
0.9842 - val_loss: 0.8797 - val_accuracy: 0.8628
Epoch 201/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0474 - accuracy:
0.9842 - val_loss: 0.8923 - val_accuracy: 0.8850
Epoch 202/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0532 - accuracy:
0.9818 - val_loss: 0.9394 - val_accuracy: 0.8717
Epoch 203/500
```

```
52/52 [==============================] - 5s 90ms/step - loss: 0.0373 - accuracy:
0.9854 - val_loss: 0.8742 - val_accuracy: 0.8805
Epoch 204/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0311 - accuracy:
0.9873 - val_loss: 0.8723 - val_accuracy: 0.8761
Epoch 205/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0296 - accuracy:
0.9909 - val_loss: 0.8359 - val_accuracy: 0.8673
Epoch 206/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0184 - accuracy:
0.9921 - val_loss: 0.8817 - val_accuracy: 0.8628
Epoch 207/500
52/52 [==============================] - 5s 94ms/step - loss: 0.0144 - accuracy:
0.9970 - val_loss: 0.9121 - val_accuracy: 0.8673
Epoch 208/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0163 - accuracy:
0.9945 - val_loss: 0.9776 - val_accuracy: 0.8673
Epoch 209/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0126 - accuracy:
0.9964 - val_loss: 0.9969 - val_accuracy: 0.8673
Epoch 210/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0274 - accuracy:
0.9873 - val_loss: 0.8649 - val_accuracy: 0.8761
Epoch 211/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0153 - accuracy:
0.9939 - val_loss: 0.9024 - val_accuracy: 0.8805
Epoch 212/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0310 - accuracy:
0.9897 - val_loss: 0.7923 - val_accuracy: 0.8805
Epoch 213/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0210 - accuracy:
0.9927 - val_loss: 0.9118 - val_accuracy: 0.8761
Epoch 214/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0134 - accuracy:
0.9951 - val_loss: 0.9297 - val_accuracy: 0.8717
Epoch 215/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0091 - accuracy:
0.9976 - val_loss: 0.9781 - val_accuracy: 0.8805
Epoch 216/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0219 - accuracy:
0.9915 - val_loss: 0.8877 - val_accuracy: 0.8850
Epoch 217/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0179 - accuracy:
0.9915 - val_loss: 0.9319 - val_accuracy: 0.8850
Epoch 218/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0243 - accuracy:
0.9885 - val_loss: 1.0389 - val_accuracy: 0.8673
Epoch 219/500
```

```
52/52 [==============================] - 5s 90ms/step - loss: 0.0322 - accuracy:
0.9885 - val_loss: 0.9698 - val_accuracy: 0.8805
Epoch 220/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0210 - accuracy:
0.9939 - val_loss: 0.9933 - val_accuracy: 0.8673
Epoch 221/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0268 - accuracy:
0.9915 - val_loss: 0.8629 - val_accuracy: 0.8894
Epoch 222/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0295 - accuracy:
0.9891 - val_loss: 0.8753 - val_accuracy: 0.8894
Epoch 223/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0459 - accuracy:
0.9860 - val_loss: 0.9222 - val_accuracy: 0.8805
Epoch 224/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0513 - accuracy:
0.9812 - val_loss: 0.7810 - val_accuracy: 0.8761
Epoch 225/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0389 - accuracy:
0.9867 - val_loss: 0.8228 - val_accuracy: 0.8628
Epoch 226/500
52/52 [==============================] - 5s 94ms/step - loss: 0.0273 - accuracy:
0.9897 - val_loss: 0.9019 - val_accuracy: 0.8805
Epoch 227/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0242 - accuracy:
0.9915 - val_loss: 0.9363 - val_accuracy: 0.8805
Epoch 228/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0238 - accuracy:
0.9891 - val_loss: 0.8413 - val_accuracy: 0.8805
Epoch 229/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0250 - accuracy:
0.9891 - val_loss: 0.8274 - val_accuracy: 0.8850
Epoch 230/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0201 - accuracy:
0.9927 - val_loss: 0.9233 - val_accuracy: 0.8894
Epoch 231/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0203 - accuracy:
0.9921 - val_loss: 1.0218 - val_accuracy: 0.8761
Epoch 232/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0189 - accuracy:
0.9933 - val_loss: 0.8438 - val_accuracy: 0.8761
Epoch 233/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0178 - accuracy:
0.9933 - val_loss: 0.9837 - val_accuracy: 0.8717
Epoch 234/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0299 - accuracy:
0.9903 - val_loss: 0.9565 - val_accuracy: 0.8894
Epoch 235/500
```

```
52/52 [==============================] - 5s 91ms/step - loss: 0.0314 - accuracy:
0.9885 - val_loss: 0.9965 - val_accuracy: 0.8673
Epoch 236/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0270 - accuracy:
0.9897 - val_loss: 0.8366 - val_accuracy: 0.8938
Epoch 237/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0305 - accuracy:
0.9879 - val_loss: 0.9004 - val_accuracy: 0.8717
Epoch 238/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0227 - accuracy:
0.9933 - val_loss: 0.8997 - val_accuracy: 0.8805
Epoch 239/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0152 - accuracy:
0.9939 - val_loss: 0.8727 - val_accuracy: 0.8938
Epoch 240/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0165 - accuracy:
0.9939 - val_loss: 0.8922 - val_accuracy: 0.8717
Epoch 241/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0154 - accuracy:
0.9927 - val_loss: 1.1190 - val_accuracy: 0.8894
Epoch 242/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0169 - accuracy:
0.9976 - val_loss: 1.0676 - val_accuracy: 0.9027
Epoch 243/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0172 - accuracy:
0.9921 - val_loss: 0.9435 - val_accuracy: 0.8761
Epoch 244/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0140 - accuracy:
0.9945 - val_loss: 0.9850 - val_accuracy: 0.8894
Epoch 245/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0154 - accuracy:
0.9964 - val_loss: 0.9544 - val_accuracy: 0.8717
Epoch 246/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0143 - accuracy:
0.9964 - val_loss: 0.9835 - val_accuracy: 0.8761
Epoch 247/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0292 - accuracy:
0.9909 - val_loss: 1.0332 - val_accuracy: 0.8628
Epoch 248/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0529 - accuracy:
0.9806 - val_loss: 0.9698 - val_accuracy: 0.8850
Epoch 249/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0263 - accuracy:
0.9897 - val_loss: 0.9804 - val_accuracy: 0.8673
Epoch 250/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0269 - accuracy:
0.9891 - val_loss: 0.9949 - val_accuracy: 0.8805
Epoch 251/500
```

```
52/52 [==============================] - 5s 91ms/step - loss: 0.0229 - accuracy:
0.9909 - val_loss: 1.0405 - val_accuracy: 0.8850
Epoch 252/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0261 - accuracy:
0.9867 - val_loss: 0.8846 - val_accuracy: 0.8938
Epoch 253/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0189 - accuracy:
0.9933 - val_loss: 1.0103 - val_accuracy: 0.8894
Epoch 254/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0210 - accuracy:
0.9933 - val_loss: 1.0075 - val_accuracy: 0.8805
Epoch 255/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0371 - accuracy:
0.9830 - val_loss: 0.8635 - val_accuracy: 0.8805
Epoch 256/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0284 - accuracy:
0.9915 - val_loss: 0.9058 - val_accuracy: 0.8805
Epoch 257/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0234 - accuracy:
0.9921 - val_loss: 1.0496 - val_accuracy: 0.8850
Epoch 258/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0272 - accuracy:
0.9897 - val_loss: 0.9949 - val_accuracy: 0.8894
Epoch 259/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0222 - accuracy:
0.9915 - val_loss: 0.9914 - val_accuracy: 0.8761
Epoch 260/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0089 - accuracy:
0.9982 - val_loss: 1.1287 - val_accuracy: 0.8717
Epoch 261/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0298 - accuracy:
0.9854 - val_loss: 1.0703 - val_accuracy: 0.8717
Epoch 262/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0293 - accuracy:
0.9891 - val_loss: 1.0132 - val_accuracy: 0.8805
Epoch 263/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0231 - accuracy:
0.9897 - val_loss: 1.0447 - val_accuracy: 0.8805
Epoch 264/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0186 - accuracy:
0.9927 - val_loss: 1.0073 - val_accuracy: 0.8805
Epoch 265/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0339 - accuracy:
0.9873 - val_loss: 1.0275 - val_accuracy: 0.8805
Epoch 266/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0192 - accuracy:
0.9903 - val_loss: 0.9939 - val_accuracy: 0.8761
Epoch 267/500
```

```
52/52 [==============================] - 5s 89ms/step - loss: 0.0249 - accuracy:
0.9927 - val_loss: 0.9878 - val_accuracy: 0.8938
Epoch 268/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0168 - accuracy:
0.9939 - val_loss: 1.0422 - val_accuracy: 0.8540
Epoch 269/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0246 - accuracy:
0.9915 - val_loss: 1.0034 - val_accuracy: 0.8761
Epoch 270/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0193 - accuracy:
0.9927 - val_loss: 1.0186 - val_accuracy: 0.8717
Epoch 271/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0251 - accuracy:
0.9903 - val_loss: 0.9473 - val_accuracy: 0.8761
Epoch 272/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0216 - accuracy:
0.9933 - val_loss: 0.9795 - val_accuracy: 0.8805
Epoch 273/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0131 - accuracy:
0.9964 - val_loss: 1.0557 - val_accuracy: 0.8894
Epoch 274/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0154 - accuracy:
0.9945 - val_loss: 1.0949 - val_accuracy: 0.8850
Epoch 275/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0157 - accuracy:
0.9933 - val_loss: 1.1485 - val_accuracy: 0.8850
Epoch 276/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0201 - accuracy:
0.9945 - val_loss: 1.0541 - val_accuracy: 0.8805
Epoch 277/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0117 - accuracy:
0.9964 - val_loss: 1.0524 - val_accuracy: 0.8761
Epoch 278/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0111 - accuracy:
0.9945 - val_loss: 1.0357 - val_accuracy: 0.8850
Epoch 279/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0155 - accuracy:
0.9939 - val_loss: 1.1128 - val_accuracy: 0.8850
Epoch 280/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0311 - accuracy:
0.9909 - val_loss: 1.1281 - val_accuracy: 0.8761
Epoch 281/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0287 - accuracy:
0.9879 - val_loss: 0.9891 - val_accuracy: 0.8894
Epoch 282/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0338 - accuracy:
0.9867 - val_loss: 1.1731 - val_accuracy: 0.8805
Epoch 283/500
```

```
52/52 [==============================] - 5s 90ms/step - loss: 0.0274 - accuracy:
0.9897 - val_loss: 0.9218 - val_accuracy: 0.8673
Epoch 284/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0162 - accuracy:
0.9958 - val_loss: 0.9705 - val_accuracy: 0.8850
Epoch 285/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0329 - accuracy:
0.9873 - val_loss: 0.9507 - val_accuracy: 0.8805
Epoch 286/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0200 - accuracy:
0.9927 - val_loss: 0.9592 - val_accuracy: 0.8805
Epoch 287/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0173 - accuracy:
0.9921 - val_loss: 1.0192 - val_accuracy: 0.8761
Epoch 288/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0191 - accuracy:
0.9927 - val_loss: 1.0012 - val_accuracy: 0.8673
Epoch 289/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0338 - accuracy:
0.9860 - val_loss: 0.8612 - val_accuracy: 0.8938
Epoch 290/500
52/52 [==============================] - 5s 88ms/step - loss: 0.0109 - accuracy:
0.9964 - val_loss: 0.9360 - val_accuracy: 0.8850
Epoch 291/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0153 - accuracy:
0.9939 - val_loss: 0.9921 - val_accuracy: 0.8673
Epoch 292/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0257 - accuracy:
0.9921 - val_loss: 1.0192 - val_accuracy: 0.8673
Epoch 293/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0139 - accuracy:
0.9939 - val_loss: 1.0358 - val_accuracy: 0.8805
Epoch 294/500
52/52 [==============================] - 5s 89ms/step - loss: 0.0157 - accuracy:
0.9945 - val_loss: 1.1709 - val_accuracy: 0.8850
Epoch 295/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0187 - accuracy:
0.9933 - val_loss: 1.1732 - val_accuracy: 0.8673
Epoch 296/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0251 - accuracy:
0.9891 - val_loss: 1.1031 - val_accuracy: 0.8850
Epoch 297/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0224 - accuracy:
0.9933 - val_loss: 1.1135 - val_accuracy: 0.8805
Epoch 298/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0240 - accuracy:
0.9921 - val_loss: 1.0201 - val_accuracy: 0.8673
Epoch 299/500
```

```
52/52 [==============================] - 5s 90ms/step - loss: 0.0285 - accuracy:
0.9891 - val_loss: 0.9305 - val_accuracy: 0.8894
Epoch 300/500
52/52 [==============================] - 5s 95ms/step - loss: 0.0306 - accuracy:
0.9891 - val_loss: 1.0012 - val_accuracy: 0.8982
Epoch 301/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0357 - accuracy:
0.9903 - val_loss: 1.1096 - val_accuracy: 0.8761
Epoch 302/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0363 - accuracy:
0.9860 - val_loss: 1.0167 - val_accuracy: 0.8850
Epoch 303/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0173 - accuracy:
0.9945 - val_loss: 0.9060 - val_accuracy: 0.8805
Epoch 304/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0260 - accuracy:
0.9885 - val_loss: 0.9306 - val_accuracy: 0.8805
Epoch 305/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0198 - accuracy:
0.9933 - val_loss: 1.1291 - val_accuracy: 0.8673
Epoch 306/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0173 - accuracy:
0.9939 - val_loss: 1.1662 - val_accuracy: 0.8805
Epoch 307/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0141 - accuracy:
0.9951 - val_loss: 0.9320 - val_accuracy: 0.8894
Epoch 308/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0253 - accuracy:
0.9903 - val_loss: 1.0697 - val_accuracy: 0.8894
Epoch 309/500
52/52 [==============================] - 5s 91ms/step - loss: 0.0319 - accuracy:
0.9903 - val_loss: 1.0607 - val_accuracy: 0.8761
Epoch 310/500
52/52 [==============================] - 5s 95ms/step - loss: 0.0346 - accuracy:
0.9867 - val_loss: 0.9244 - val_accuracy: 0.8673
Epoch 311/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0252 - accuracy:
0.9891 - val_loss: 1.1671 - val_accuracy: 0.8761
Epoch 312/500
52/52 [==============================] - 5s 92ms/step - loss: 0.0234 - accuracy:
0.9891 - val_loss: 1.1984 - val_accuracy: 0.8540
Epoch 313/500
52/52 [==============================] - 5s 90ms/step - loss: 0.0177 - accuracy:
0.9933 - val_loss: 1.0214 - val_accuracy: 0.8761
Epoch 314/500
52/52 [==============================] - 5s 93ms/step - loss: 0.0176 - accuracy:
0.9933 - val_loss: 1.0822 - val_accuracy: 0.8673
Epoch 315/500
```

```
52/52 [==============================] - 5s 93ms/step - loss: 0.0186 - accuracy:
0.9939 - val_loss: 1.1553 - val_accuracy: 0.8540
Number of epochs run: 315
Best epoch according to callback: 14
Argmin of val_loss: 14
```

[19]:
```python
model.save("cats_feature_extractor.keras")
```

[20]:
```python
# Fine tuning the top of InceptionV3
base_model.trainable = True

fine_tune_from = 240

for layer in base_model.layers[:fine_tune_from]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

fine_tune_history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=5
)
```

```
Epoch 1/5
52/52 [==============================] - 25s 235ms/step - loss: 0.1616 -
accuracy: 0.9430 - val_loss: 0.3349 - val_accuracy: 0.8982
Epoch 2/5
52/52 [==============================] - 5s 105ms/step - loss: 0.0767 -
accuracy: 0.9788 - val_loss: 0.3618 - val_accuracy: 0.8982
Epoch 3/5
52/52 [==============================] - 5s 104ms/step - loss: 0.0640 -
accuracy: 0.9836 - val_loss: 0.3546 - val_accuracy: 0.8982
Epoch 4/5
52/52 [==============================] - 5s 105ms/step - loss: 0.0419 -
accuracy: 0.9927 - val_loss: 0.3768 - val_accuracy: 0.9071
Epoch 5/5
52/52 [==============================] - 5s 105ms/step - loss: 0.0305 -
accuracy: 0.9921 - val_loss: 0.3938 - val_accuracy: 0.9027
```

12 MANUAL TEST

[21]:
```
def TestImage(imgpath):
    img = tf.keras.utils.load_img(imgpath, target_size=IMG_SIZE)

    img_array = tf.keras.utils.img_to_array(img)
    img_array = tf.expand_dims(img_array, 0)

    predictions = model.predict(img_array)[0]

    for name, p in sorted(zip(class_names, predictions), key=lambda x: -x[1]):
        print(f"{name:20s} {p:.3f}")
```

[22]:
```
import glob
import random

def getRandTestImage(breed):
    return random.choice(glob.glob(f"CatData/_tmp_standardized/test/{breed}/*"))
```

[23]:
```
TestImage(getRandTestImage("ragdoll"))
```

```
1/1 [==============================] - 2s 2s/step
ragdoll              0.999
maine_coon           0.001
domestic_shorthair   0.000
siamese              0.000
bengal               0.000
```

[24]:
```
TestImage(getRandTestImage("maine_coon"))
```

```
1/1 [==============================] - 0s 41ms/step
maine_coon           0.972
domestic_shorthair   0.023
ragdoll              0.006
bengal               0.000
siamese              0.000
```

[25]:
```
TestImage(getRandTestImage("bengal"))
```

```
1/1 [==============================] - 0s 27ms/step
bengal               0.710
domestic_shorthair   0.281
maine_coon           0.009
siamese              0.000
ragdoll              0.000
```

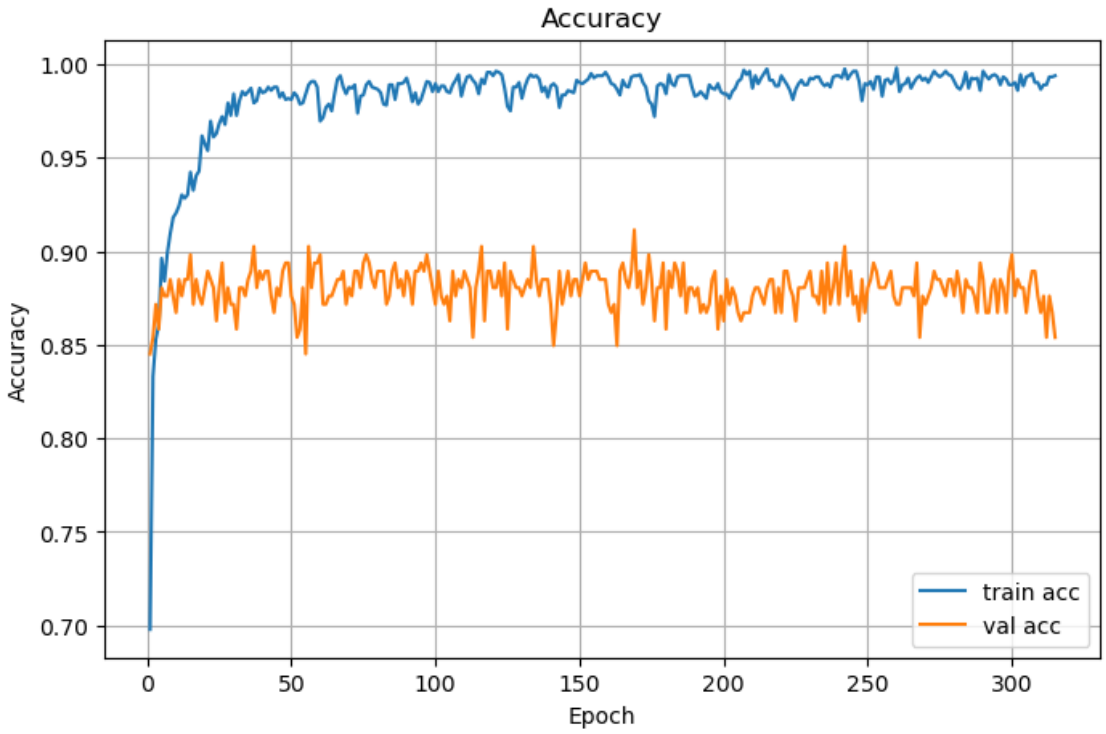
[26]:
```
TestImage(getRandTestImage("siamese"))
```

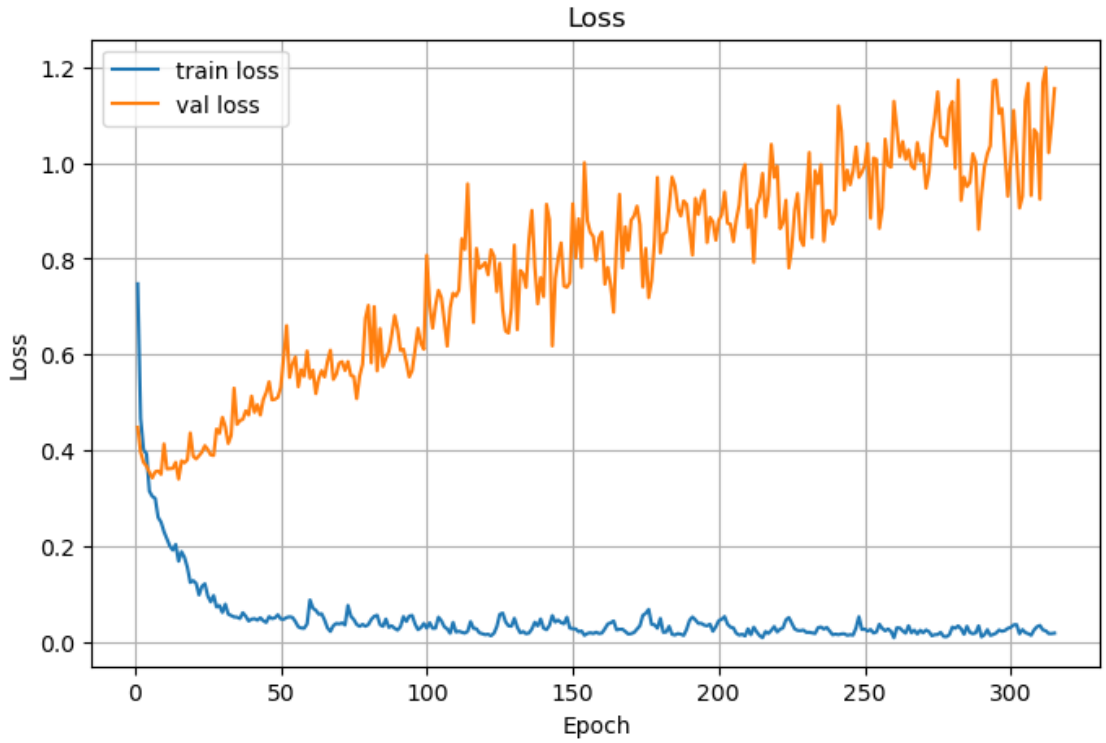
```
1/1 [==============================] - 0s 27ms/step
siamese              1.000
ragdoll              0.000
domestic_shorthair   0.000
bengal               0.000
maine_coon           0.000
```

[27]:
```
TestImage(getRandTestImage("domestic_shorthair"))
```

```
1/1 [==============================] - 0s 26ms/step
domestic_shorthair   0.998
maine_coon           0.002
bengal               0.000
ragdoll              0.000
siamese              0.000
```

[28]:
```
import matplotlib.pyplot as plt

def plot_history(history):
    acc = history.history["accuracy"]
    val_acc = history.history["val_accuracy"]
    loss = history.history["loss"]
    val_loss = history.history["val_loss"]

    epochs = range(1, len(acc) + 1)

    plt.figure()
    plt.plot(epochs, acc, label="train acc")
    plt.plot(epochs, val_acc, label="val acc")
    plt.legend()
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.title("Accuracy")

    plt.figure()
    plt.plot(epochs, loss, label="train loss")
    plt.plot(epochs, val_loss, label="val loss")
    plt.legend()
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("Loss")

plot_history(history)
```


Accuracy
1.00
0.95
0.90
0.85
0.80
0.75
0.70
Accuracy
0
50
100
150
200
250
300
Epoch
train acc
val acc


Loss
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Loss
0
50
100
150
200
250
300
Epoch
train loss
val loss

[29]:
```
len(history.history["loss"])
```

[29]: 315

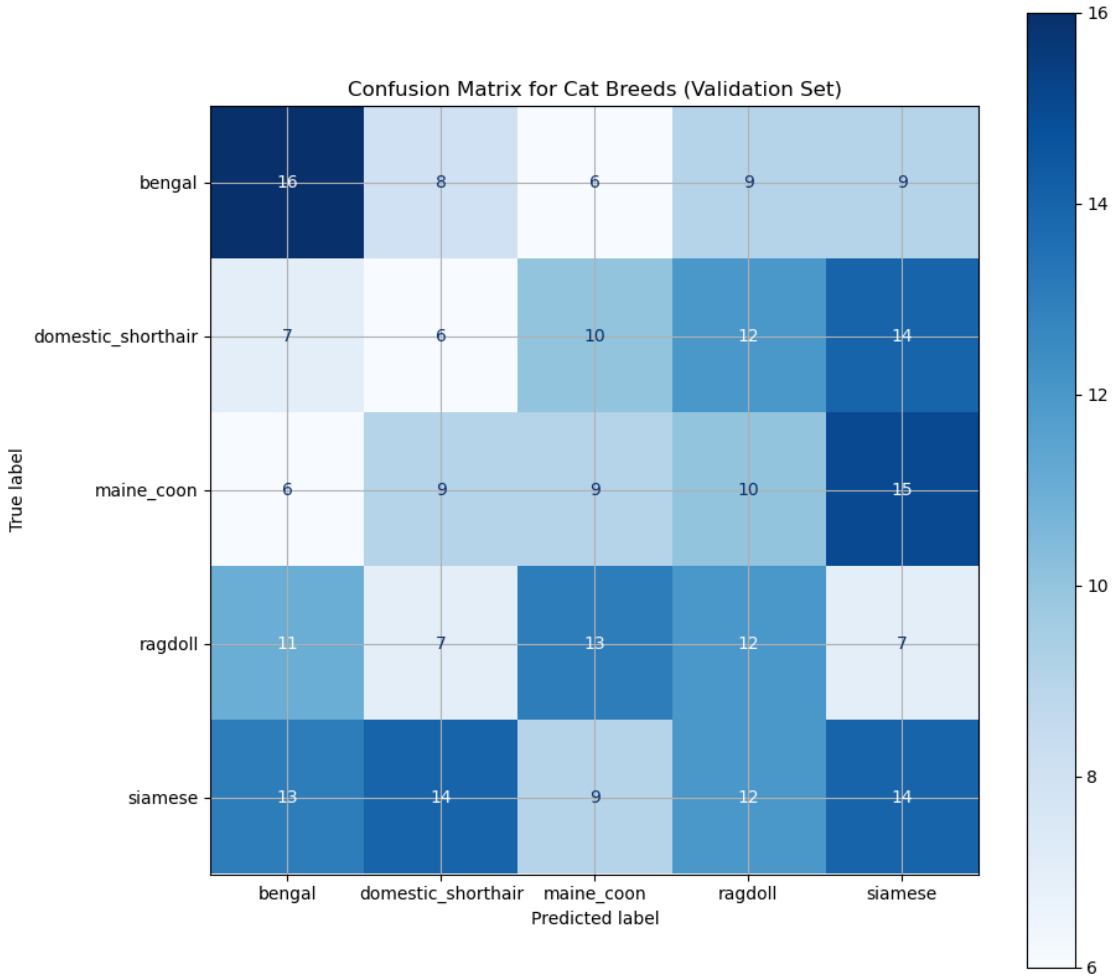
[30]:
```
print("Best epoch:", early_stop.best_epoch)
```

```
Best epoch: 14
```

[31]:
```
import sklearn

# 1. Hent Sande Etiketter (True Labels)
# Samler alle 'y' værdier (labels) fra valideringssættet.
y_true = []
for images, labels in test_ds:
    # labels.numpy() konverterer tf.Tensor til NumPy array
    y_true.extend(labels.numpy())

y_true = np.array(y_true)
print(f"y_true shape: {y_true.shape}")

# 2. Hent Modellens Rå Forudsigelser (Probabilities)
# model.predict tager hele val_ds datasættet som input
y_pred_raw = model.predict(test_ds)
print(f"y_pred_raw shape: {y_pred_raw.shape}")
# Bør være (antal billeder, 5 racer)

# 3. Konverter Sandsynligheder til Klasser
# np.argmax vælger indexet (klassenummeret) med den højeste sandsynlighed
y_pred = np.argmax(y_pred_raw, axis=1)
print(f"y_pred shape: {y_pred.shape}")

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Kør de ovenstående trin 1 og 2 for at få y_true og y_pred...

# 4. Generer Matricen
cm = confusion_matrix(y_true, y_pred)

# 5. Visualiser Matricen
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names # class_names fik du fra val_ds
)
fig, ax = plt.subplots(figsize=(10, 10))
disp.plot(ax=ax, cmap=plt.cm.Blues)
plt.title("Confusion Matrix for Cat Breeds (Validation Set)")
```

```
plt.show()
```

```
y_true shape: (258,)
9/9 [==============================] - 1s 72ms/step
y_pred_raw shape: (258, 5)
y_pred shape: (258,)
```



[ ]:

[ ]:

[ ]:

[ ]: